



**ĐẠI HỌC
BÁCH KHOA HÀ NỘI**
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Lingine

Kỳ thủ Cờ Tướng nhân tạo sử dụng tìm kiếm đối kháng

Trần Tuấn Anh – 202416124

Lê Thành Trung – 202400076

Bùi Tiến Dũng – 202416167

Hà Nội, Ngày 20 tháng 6 năm 2026

ONE LOVE. ONE FUTURE.

Bài Toán

Biểu Diễn Trạng Thái

Sinh Nước Đi

Hàm Đánh Giá

Tìm Kiếm

Thực Nghiệm Và Demo

Kết luận

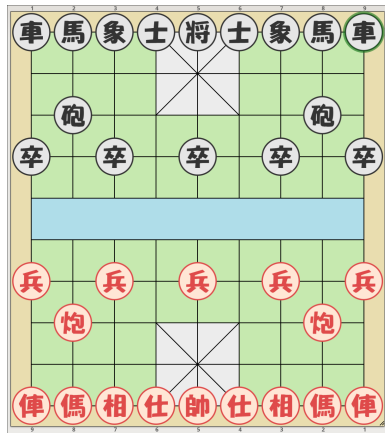
Biểu Diễn Trạng Thái

Hàm Đánh Giá

Thực Nghiệm Và Demo

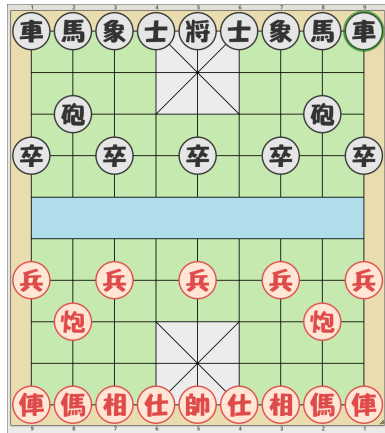
Kết luận

- ◇ Xây dựng AI chơi Cờ Tướng.



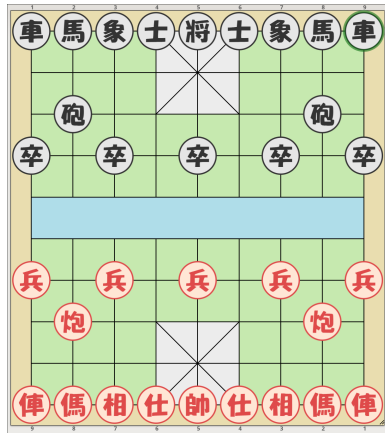
Hình 1: Bàn cờ khởi đầu.

- ◇ Xây dựng AI chơi Cờ Tướng.
- ◇ AI được mô hình hóa như một Intelligent Agent.



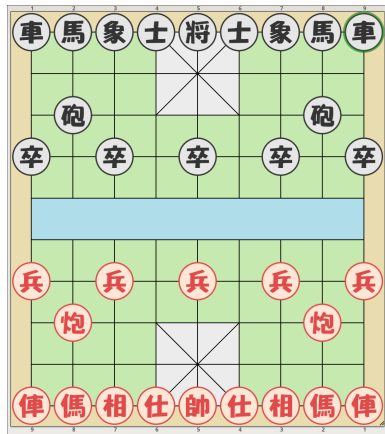
Hình 1: Bàn cờ khởi đầu.

- ◇ Xây dựng AI chơi Cờ Tướng.
- ◇ AI được mô hình hóa như một Intelligent Agent.
- ◇ Quan sát và phân tích trạng thái bàn cờ.



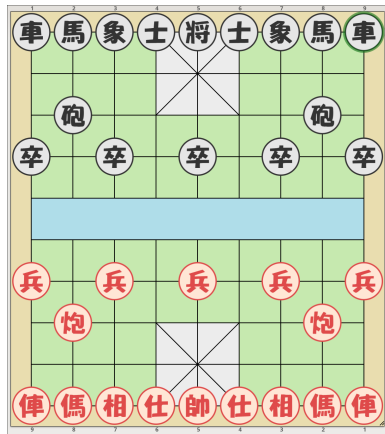
Hình 1: Bàn cờ khởi đầu.

- ◇ Xây dựng AI chơi Cờ Tướng.
- ◇ AI được mô hình hóa như một Intelligent Agent.
- ◇ Quan sát và phân tích trạng thái bàn cờ.
- ◇ Đưa ra nước đi tốt nhất trong môi trường đối kháng.



Hình 1: Bàn cờ khởi đầu.

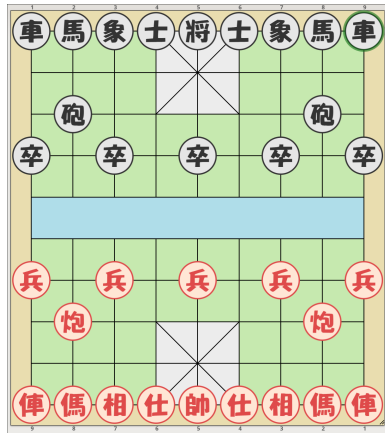
- ◇ Xây dựng AI chơi Cờ Tướng.
- ◇ AI được mô hình hóa như một Intelligent Agent.
- ◇ Quan sát và phân tích trạng thái bàn cờ.
- ◇ Đưa ra nước đi tốt nhất trong môi trường đối kháng.
- ◇ Một trong những bài toán kinh điển của AI: Game playing.



Hình 1: Bàn cờ khởi đầu.

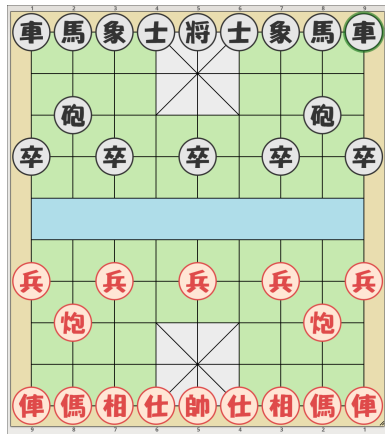
◇ Cờ Tướng là bài toán:

1. Hai người chơi.
2. Đối kháng Zero-Sum.



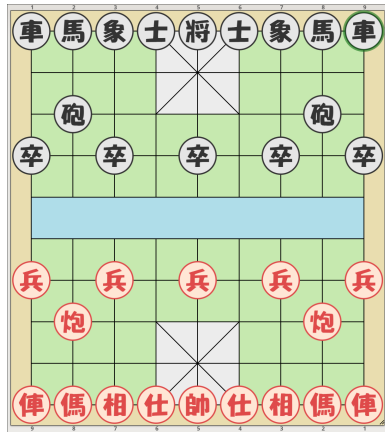
Hình 1: Bàn cờ khởi đầu.

- ◇ Cờ Tướng là bài toán:
 1. Hai người chơi.
 2. Đối kháng Zero-Sum.
- ◇ Không gian tìm kiếm rất lớn ($\approx 10^{150}$)



Hình 1: Bàn cờ khởi đầu.

- ◇ Cờ Tướng là bài toán:
 1. Hai người chơi.
 2. Đối kháng Zero-Sum.
- ◇ Không gian tìm kiếm rất lớn ($\approx 10^{150}$)
- ◇ Vì vậy, đòi hỏi:
 1. Tìm kiếm thông minh.
 2. Đánh giá chính xác.
 3. Hiệu năng cao.



Hình 1: Bàn cờ khởi đầu.

Agent Đầu vào



Hình 2: Sơ đồ kiến trúc của Lngine

Agent

Đầu vào

- ◇ Trạng thái hiện tại của bàn cờ.

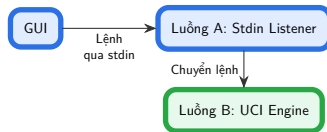


Hình 2: Sơ đồ kiến trúc của Lngine

Agent

Đầu vào

- ◇ Trạng thái hiện tại của bàn cờ.
- ◇ Lượt chơi hiện tại.

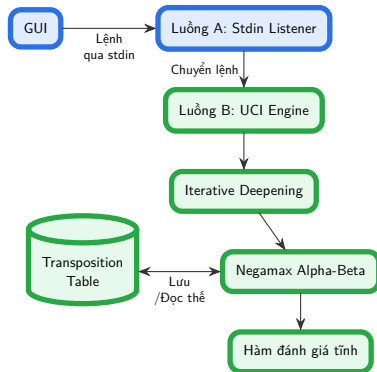


Hình 2: Sơ đồ kiến trúc của Lngine

Agent

Đầu vào

- ◇ Trạng thái hiện tại của bàn cờ.
- ◇ Lượt chơi hiện tại.
- ◇ Chuỗi các nước đi.



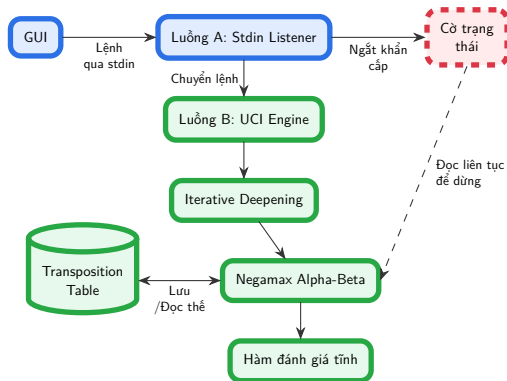
Hình 2: Sơ đồ kiến trúc của Lingine

Agent

Đầu vào

- ◇ Trạng thái hiện tại của bàn cờ.
- ◇ Lượt chơi hiện tại.
- ◇ Chuỗi các nước đi.

Đầu ra



Hình 2: Sơ đồ kiến trúc của Lingine

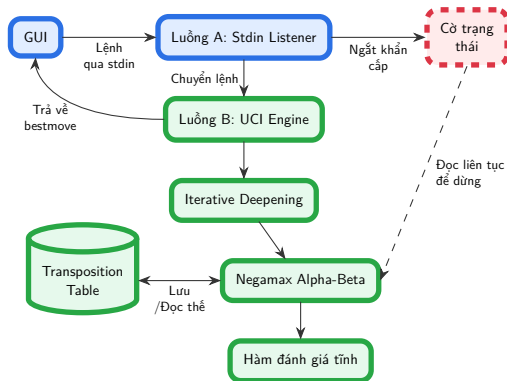
Agent

Đầu vào

- ◇ Trạng thái hiện tại của bàn cờ.
- ◇ Lượt chơi hiện tại.
- ◇ Chuỗi các nước đi.

Đầu ra

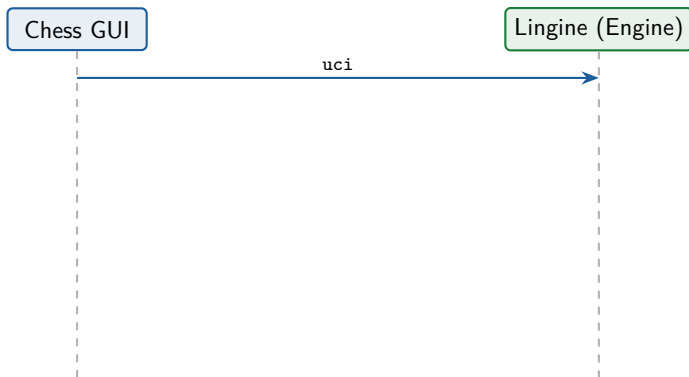
- ◇ Nước đi tốt nhất cho trạng thái bàn cờ hiện tại.



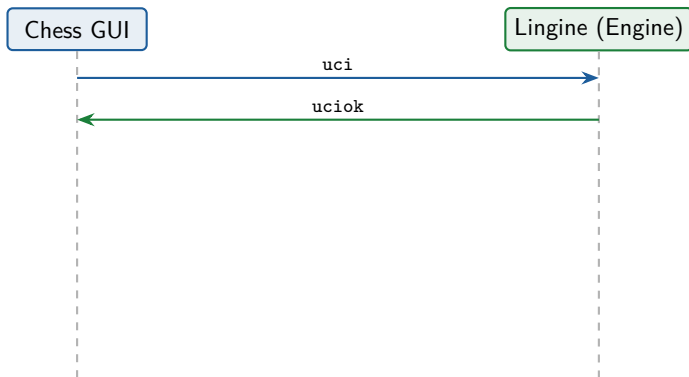
Hình 2: Sơ đồ kiến trúc của Lingine



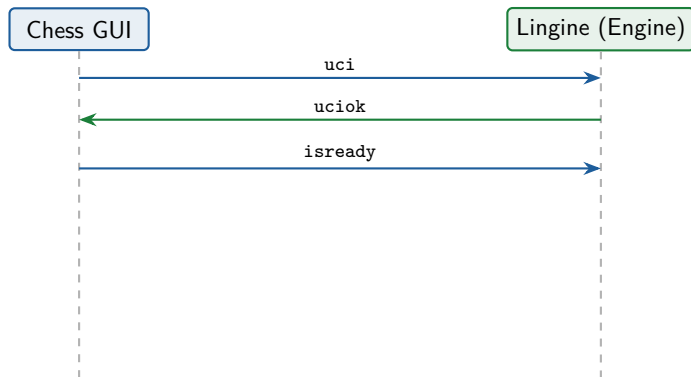
Hình 3: Luồng giao tiếp qua giao thức UCI



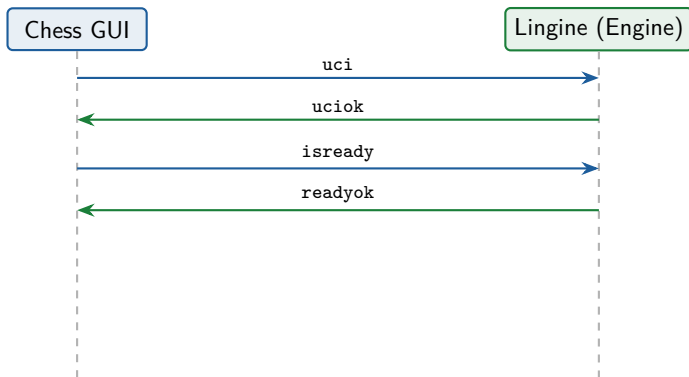
Hình 3: Luồng giao tiếp qua giao thức UCI



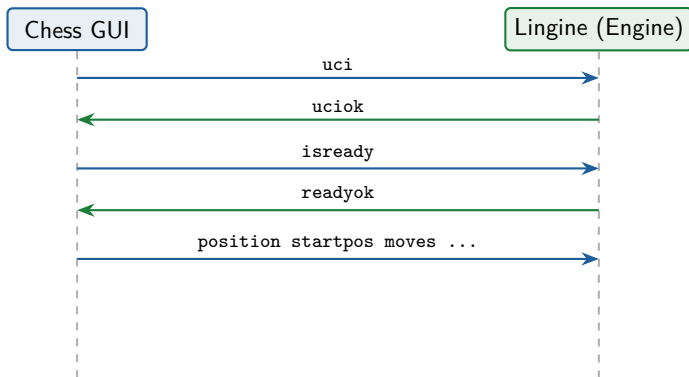
Hình 3: Luồng giao tiếp qua giao thức UCI



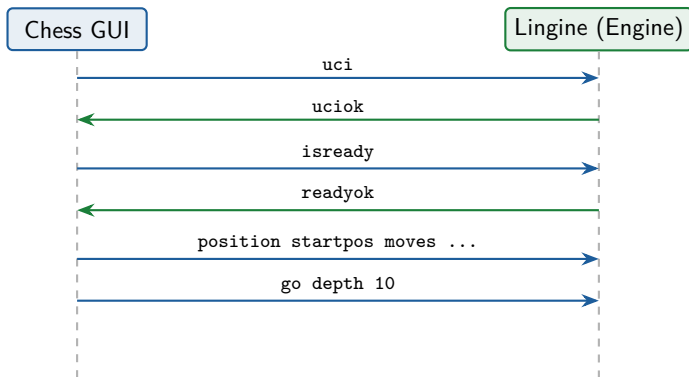
Hình 3: Luồng giao tiếp qua giao thức UCI



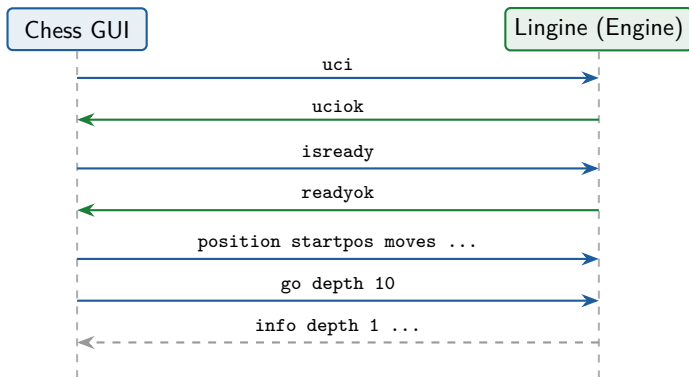
Hình 3: Luồng giao tiếp qua giao thức UCI



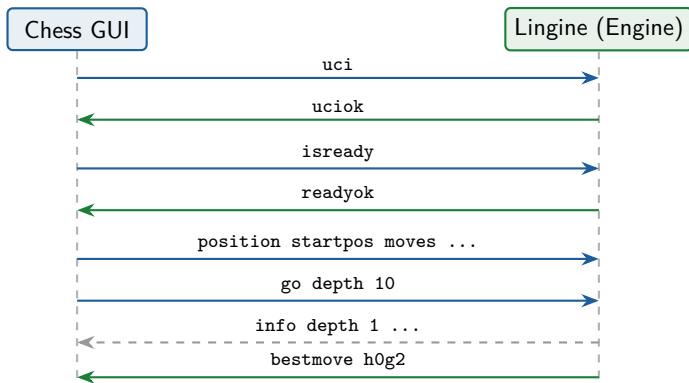
Hình 3: Luồng giao tiếp qua giao thức UCI



Hình 3: Luồng giao tiếp qua giao thức UCI



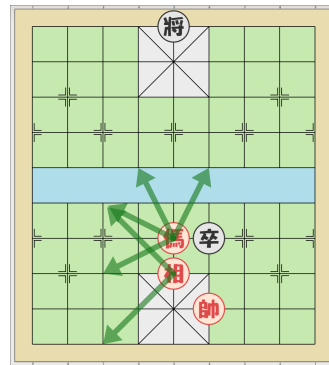
Hình 3: Luồng giao tiếp qua giao thức UCI



Hình 3: Luồng giao tiếp qua giao thức UCI

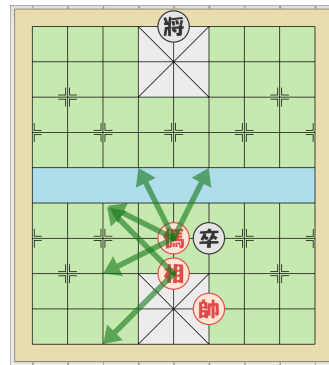
Cấu trúc

Đặc thù



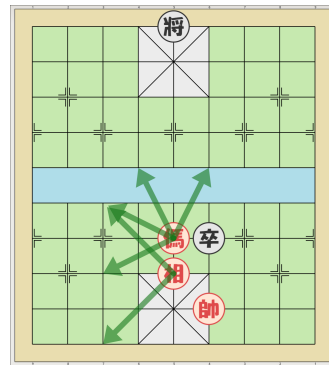
Hình 4: Cản mắt tượng và chân mã.

Cấu trúc	Đặc thù
90 ô	Không gian tìm kiếm rất lớn



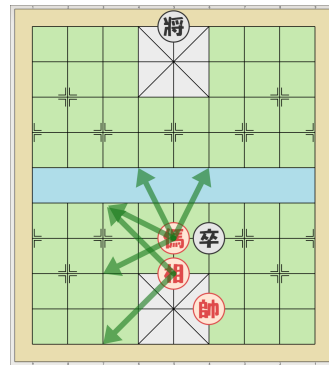
Hình 4: Cản mắt tượng và chân mã.

Cấu trúc	Đặc thù
90 ô	Không gian tìm kiếm rất lớn
Pháo	Nước đi phụ thuộc quân cản



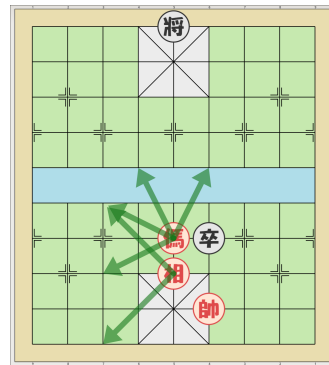
Hình 4: Cản mắt tượng và chân mã.

Cấu trúc	Đặc thù
90 ô	Không gian tìm kiếm rất lớn
Pháo	Nước đi phụ thuộc quân cản
Cản chân	Giảm sự linh hoạt của Mã và Tượng



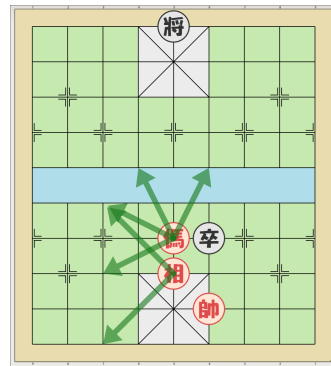
Hình 4: Cản mắt tượng và chân mã.

Cấu trúc	Đặc thù
90 ô	Không gian tìm kiếm rất lớn
Pháo	Nước đi phụ thuộc quân cản
Cản chân	Giảm sự linh hoạt của Mã và Tượng
Hàm đánh giá	Phức tạp, yêu cầu nhiều kiến thức cho tàn cuộc



Hình 4: Cản mắt tượng và chân mã.

Cấu trúc	Đặc thù
90 ô	Không gian tìm kiếm rất lớn
Pháo	Nước đi phụ thuộc quân cản
Cản chân	Giảm sự linh hoạt của Mã và Tượng
Hàm đánh giá	Phức tạp, yêu cầu nhiều kiến thức cho tàn cuộc
Bộ luật đặc biệt	Lộ mặt Tướng, chiếu/đuổi dai



Hình 4: Cản mắt tướng và chân mã.

Bài Toán

Biểu Diễn Trạng Thái

Sinh Nước Đi

Hàm Đánh Giá

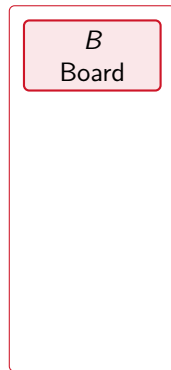
Tìm Kiếm

Thực Nghiệm Và Demo

Kết luận

$$S = \langle B, p, c_h, h \rangle$$

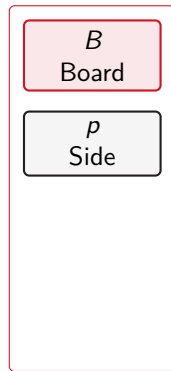
◇ B : bàn cờ 10×9 .



Hình 5: Không gian trạng thái

$$S = \langle B, p, c_h, h \rangle$$

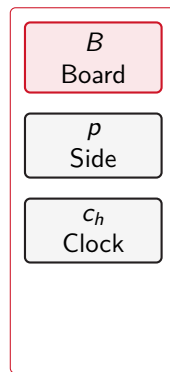
- ◇ B : bàn cờ 10×9 .
- ◇ p : bên đi.



Hình 5: Không gian trạng thái

$$S = \langle B, p, c_h, h \rangle$$

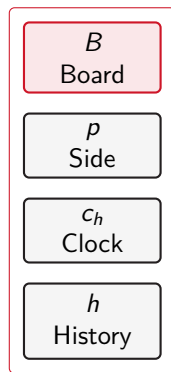
- ◇ B : bàn cờ 10×9 .
- ◇ p : bên đi.
- ◇ c_h : bộ đếm luật 60 nước.



Hình 5: Không gian trạng thái

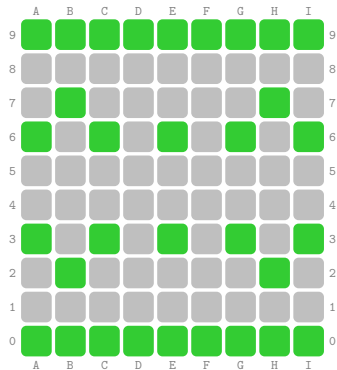
$$S = \langle B, p, c_h, h \rangle$$

- ◇ B : bàn cờ 10×9 .
- ◇ p : bên đi.
- ◇ c_h : bộ đếm luật 60 nước.
- ◇ h : lịch sử để undo và phát hiện lặp.



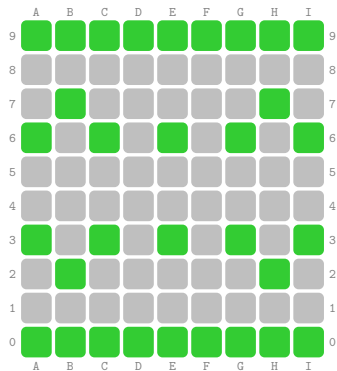
Hình 5: Không gian trạng thái

- ◇ Cờ tướng có 90 ô nên u64 không đủ.



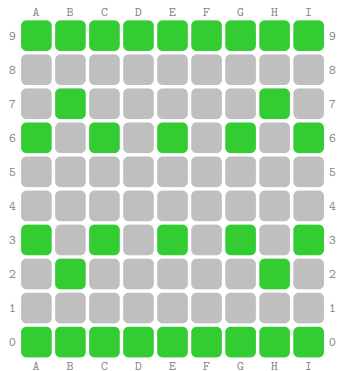
Hình 6: Bitboard cho Bàn cờ tướng bắt đầu

- ◇ Cờ tướng có 90 ô nên $\mathbf{u64}$ không đủ.
- ◇ Lingine dùng $\mathbf{u128}$; 90 bit đầu biểu diễn 90 ô.



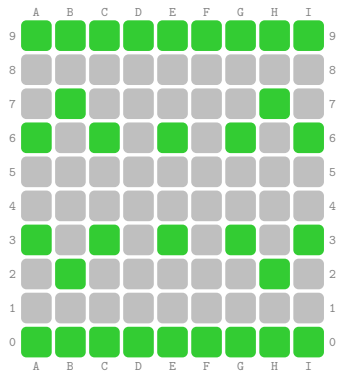
Hình 6: Bitboard cho Bàn cờ tướng bắt đầu

- ◇ Cờ tướng có 90 ô nên $\mathbf{u64}$ không đủ.
- ◇ Lingine dùng $\mathbf{u128}$; 90 bit đầu biểu diễn 90 ô.
- ◇ Bitboard theo màu và theo loại quân giúp query nhanh.



Hình 6: Bitboard cho Bàn cờ tướng bắt đầu

- ◇ Cờ tướng có 90 ô nên u_{64} không đủ.
- ◇ Lingine dùng u_{128} ; 90 bit đầu biểu diễn 90 ô.
- ◇ Bitboard theo màu và theo loại quân giúp query nhanh.
- ◇ Có thể tìm chỉ số của những bit bật bằng instruction TZCNT của $x86_64$ chỉ tốn vài chu kỳ CPU.



Hình 6: Bitboard cho Bàn cờ tướng bắt đầu

Ví dụ truy vấn tìm kiếm quân xe đỏ

	A	B	C	D	E	F	G	H	I	
9	81	82	83	84	85	86	87	88	89	9
8	72	73	74	75	76	77	78	79	80	8
7	63	64	65	66	67	68	69	70	71	7
6	54	55	56	57	58	59	60	61	62	6
5	45	46	47	48	49	50	51	52	53	5
4	36	37	38	39	40	41	42	43	44	4
3	27	28	29	30	31	32	33	34	35	3
2	18	19	20	21	22	23	24	25	26	2
1	9	10	11	12	13	14	15	16	17	1
0	0	1	2	3	4	5	6	7	8	0
	A	B	C	D	E	F	G	H	I	

Bitboard cho các quân
xe

Ví dụ truy vấn tìm kiếm quân xe đỏ

	A	B	C	D	E	F	G	H	I	
9	81	82	83	84	85	86	87	88	89	9
8	72	73	74	75	76	77	78	79	80	8
7	63	64	65	66	67	68	69	70	71	7
6	54	55	56	57	58	59	60	61	62	6
5	45	46	47	48	49	50	51	52	53	5
4	36	37	38	39	40	41	42	43	44	4
3	27	28	29	30	31	32	33	34	35	3
2	18	19	20	21	22	23	24	25	26	2
1	9	10	11	12	13	14	15	16	17	1
0	0	1	2	3	4	5	6	7	8	0
	A	B	C	D	E	F	G	H	I	

&

Bitboard cho các quân
xe

Ví dụ truy vấn tìm kiếm quân xe đỏ

	A	B	C	D	E	F	G	H	I	
9	81	82	83	84	85	86	87	88	89	9
8	72	73	74	75	76	77	78	79	80	8
7	63	64	65	66	67	68	69	70	71	7
6	54	55	56	57	58	59	60	61	62	6
5	45	46	47	48	49	50	51	52	53	5
4	36	37	38	39	40	41	42	43	44	4
3	27	28	29	30	31	32	33	34	35	3
2	18	19	20	21	22	23	24	25	26	2
1	9	10	11	12	13	14	15	16	17	1
0	0	1	2	3	4	5	6	7	8	0
	A	B	C	D	E	F	G	H	I	

&

	A	B	C	D	E	F	G	H	I	
9	81	82	83	84	85	86	87	88	89	9
8	72	73	74	75	76	77	78	79	80	8
7	63	64	65	66	67	68	69	70	71	7
6	54	55	56	57	58	59	60	61	62	6
5	45	46	47	48	49	50	51	52	53	5
4	36	37	38	39	40	41	42	43	44	4
3	27	28	29	30	31	32	33	34	35	3
2	18	19	20	21	22	23	24	25	26	2
1	9	10	11	12	13	14	15	16	17	1
0	0	1	2	3	4	5	6	7	8	0
	A	B	C	D	E	F	G	H	I	

Bitboard cho các quân
xe

Bitboard cho các quân
bên đỏ

Ví dụ truy vấn tìm kiếm quân xe đỏ

	A	B	C	D	E	F	G	H	I	
9	81	82	83	84	85	86	87	88	89	9
8	72	73	74	75	76	77	78	79	80	8
7	63	64	65	66	67	68	69	70	71	7
6	54	55	56	57	58	59	60	61	62	6
5	45	46	47	48	49	50	51	52	53	5
4	36	37	38	39	40	41	42	43	44	4
3	27	28	29	30	31	32	33	34	35	3
2	18	19	20	21	22	23	24	25	26	2
1	9	10	11	12	13	14	15	16	17	1
0	0	1	2	3	4	5	6	7	8	0
	A	B	C	D	E	F	G	H	I	

&

	A	B	C	D	E	F	G	H	I	
9	81	82	83	84	85	86	87	88	89	9
8	72	73	74	75	76	77	78	79	80	8
7	63	64	65	66	67	68	69	70	71	7
6	54	55	56	57	58	59	60	61	62	6
5	45	46	47	48	49	50	51	52	53	5
4	36	37	38	39	40	41	42	43	44	4
3	27	28	29	30	31	32	33	34	35	3
2	18	19	20	21	22	23	24	25	26	2
1	9	10	11	12	13	14	15	16	17	1
0	0	1	2	3	4	5	6	7	8	0
	A	B	C	D	E	F	G	H	I	

=

Bitboard cho các quân
xe

Bitboard cho các quân
bên đỏ

Ví dụ truy vấn tìm kiếm quân xe đỏ

	A	B	C	D	E	F	G	H	I	
9	81	82	83	84	85	86	87	88	89	9
8	72	73	74	75	76	77	78	79	80	8
7	63	64	65	66	67	68	69	70	71	7
6	54	55	56	57	58	59	60	61	62	6
5	45	46	47	48	49	50	51	52	53	5
4	36	37	38	39	40	41	42	43	44	4
3	27	28	29	30	31	32	33	34	35	3
2	18	19	20	21	22	23	24	25	26	2
1	9	10	11	12	13	14	15	16	17	1
0	0	1	2	3	4	5	6	7	8	0
	A	B	C	D	E	F	G	H	I	

Bitboard cho các quân
xe

&

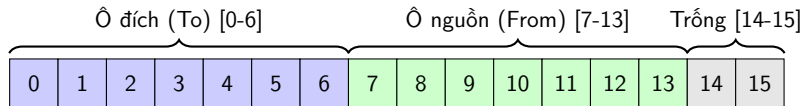
	A	B	C	D	E	F	G	H	I	
9	81	82	83	84	85	86	87	88	89	9
8	72	73	74	75	76	77	78	79	80	8
7	63	64	65	66	67	68	69	70	71	7
6	54	55	56	57	58	59	60	61	62	6
5	45	46	47	48	49	50	51	52	53	5
4	36	37	38	39	40	41	42	43	44	4
3	27	28	29	30	31	32	33	34	35	3
2	18	19	20	21	22	23	24	25	26	2
1	9	10	11	12	13	14	15	16	17	1
0	0	1	2	3	4	5	6	7	8	0
	A	B	C	D	E	F	G	H	I	

Bitboard cho các quân
bên đỏ

=

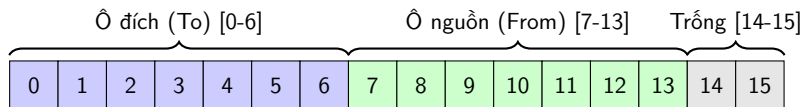
	A	B	C	D	E	F	G	H	I	
9	81	82	83	84	85	86	87	88	89	9
8	72	73	74	75	76	77	78	79	80	8
7	63	64	65	66	67	68	69	70	71	7
6	54	55	56	57	58	59	60	61	62	6
5	45	46	47	48	49	50	51	52	53	5
4	36	37	38	39	40	41	42	43	44	4
3	27	28	29	30	31	32	33	34	35	3
2	18	19	20	21	22	23	24	25	26	2
1	9	10	11	12	13	14	15	16	17	1
0	0	1	2	3	4	5	6	7	8	0
	A	B	C	D	E	F	G	H	I	

Bitboard cho các quân
xe bên đỏ



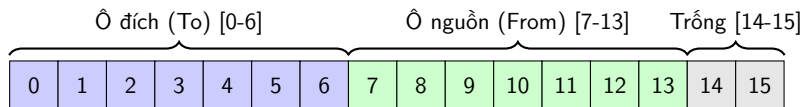
Hình 7: Sơ đồ khối bit-field minh họa dữ liệu nước đi 16-bit trong Lingine

◇ Kiểu Move là NonZeroU16.



Hình 7: Sơ đồ khối bit-field minh họa dữ liệu nước đi 16-bit trong Lingine

- ◇ Kiểu Move là NonZeroU16.
- ◇ Nhỏ, truyền theo value, hợp cache.



Hình 7: Sơ đồ khối bit-field minh họa dữ liệu nước đi 16-bit trong Lingine

- ◇ Kiểu Move là NonZeroU16.
- ◇ Nhỏ, truyền theo value, hợp cache.
- ◇ Phù hợp khi search cần duyệt hàng triệu nước đi.

Mã băm ban đầu: *hash*

$$\text{Hash}(S_0) = Z_{\text{side}} \oplus \left(\bigoplus_{sq=0}^{89} Z_{\text{piece}(sq),sq} \right)$$

Trong đó:

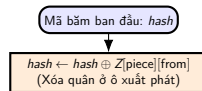
- ◇ Z_{side} là 0 nếu *side* là Đỏ, ngược lại là một giá trị ngẫu nhiên.
- ◇ $Z_{\text{piece}(sq),sq}$ là giá trị ngẫu nhiên ứng với quân cờ đang ở ô *sq*. Nếu ô *sq* trống thì giá trị này bằng 0.

Hình 8: Sơ đồ luồng cập nhật vi phân Zobrist Hash sử dụng toán tử logic $\oplus$

$$\text{Hash}(S_0) = Z_{\text{side}} \oplus \left(\bigoplus_{sq=0}^{89} Z_{\text{piece}(sq),sq} \right)$$

Trong đó:

- ◇ Z_{side} là 0 nếu side là Đỏ, ngược lại là một giá trị ngẫu nhiên.
- ◇ $Z_{\text{piece}(sq),sq}$ là giá trị ngẫu nhiên ứng với quân cờ đang ở ô sq . Nếu ô sq trống thì giá trị này bằng 0.

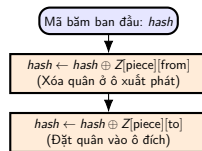


Hình 8: Sơ đồ luồng cập nhật vi phân Zobrist Hash sử dụng toán tử logic $\oplus$

$$\text{Hash}(S_0) = Z_{\text{side}} \oplus \left(\bigoplus_{sq=0}^{89} Z_{\text{piece}(sq),sq} \right)$$

Trong đó:

- ◇ Z_{side} là 0 nếu side là Đỏ, ngược lại là một giá trị ngẫu nhiên.
- ◇ $Z_{\text{piece}(sq),sq}$ là giá trị ngẫu nhiên ứng với quân cờ đang ở ô sq . Nếu ô sq trống thì giá trị này bằng 0.

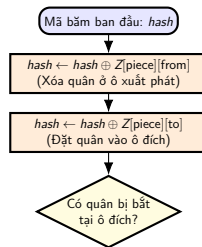


Hình 8: Sơ đồ luồng cập nhật vi phân Zobrist Hash sử dụng toán tử logic $\oplus$

$$\text{Hash}(S_0) = Z_{\text{side}} \oplus \left(\bigoplus_{sq=0}^{89} Z_{\text{piece}(sq),sq} \right)$$

Trong đó:

- ◇ Z_{side} là 0 nếu side là Đỏ, ngược lại là một giá trị ngẫu nhiên.
- ◇ $Z_{\text{piece}(sq),sq}$ là giá trị ngẫu nhiên ứng với quân cờ đang ở ô sq . Nếu ô sq trống thì giá trị này bằng 0.

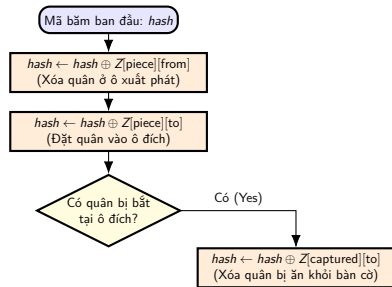


Hình 8: Sơ đồ luồng cập nhật vi phân Zobrist Hash sử dụng toán tử logic $\oplus$

$$\text{Hash}(S_0) = Z_{\text{side}} \oplus \left(\bigoplus_{sq=0}^{89} Z_{\text{piece}(sq),sq} \right)$$

Trong đó:

- ◇ Z_{side} là 0 nếu *side* là Đỏ, ngược lại là một giá trị ngẫu nhiên.
- ◇ $Z_{\text{piece}(sq),sq}$ là giá trị ngẫu nhiên ứng với quân cờ đang ở ô sq . Nếu ô sq trống thì giá trị này bằng 0.

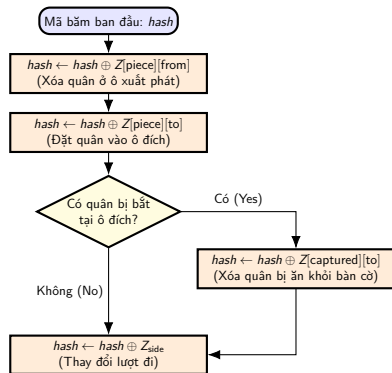


Hình 8: Sơ đồ luồng cập nhật vi phân Zobrist Hash sử dụng toán tử logic $\oplus$

$$\text{Hash}(S_0) = Z_{\text{side}} \oplus \left(\bigoplus_{sq=0}^{89} Z_{\text{piece}(sq),sq} \right)$$

Trong đó:

- ◇ Z_{side} là 0 nếu side là Đỏ, ngược lại là một giá trị ngẫu nhiên.
- ◇ $Z_{\text{piece}(sq),sq}$ là giá trị ngẫu nhiên ứng với quân cờ đang ở ô sq . Nếu ô sq trống thì giá trị này bằng 0.

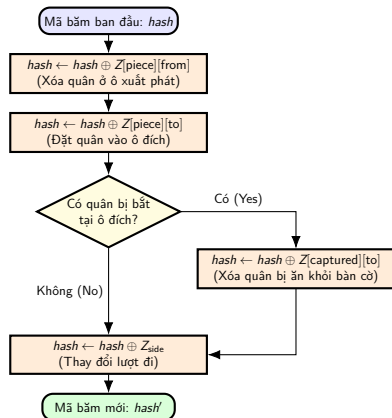


Hình 8: Sơ đồ luồng cập nhật vi phân Zobrist Hash sử dụng toán tử logic $\oplus$

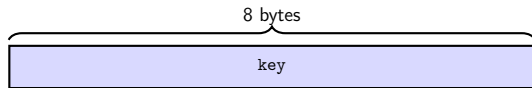
$$\text{Hash}(S_0) = Z_{\text{side}} \oplus \left(\bigoplus_{sq=0}^{89} Z_{\text{piece}(sq),sq} \right)$$

Trong đó:

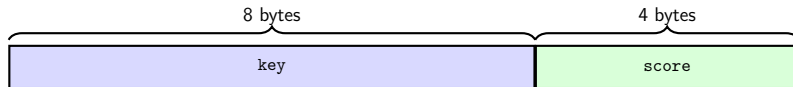
- ◇ Z_{side} là 0 nếu side là Đỏ, ngược lại là một giá trị ngẫu nhiên.
- ◇ $Z_{\text{piece}(sq),sq}$ là giá trị ngẫu nhiên ứng với quân cờ đang ở ô sq . Nếu ô sq trống thì giá trị này bằng 0.



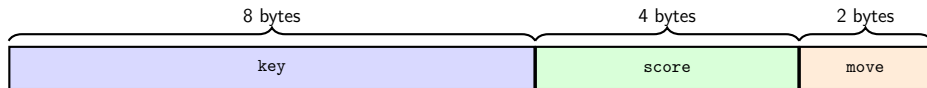
Hình 8: Sơ đồ luồng cập nhật vi phân Zobrist Hash sử dụng toán tử logic $\oplus$



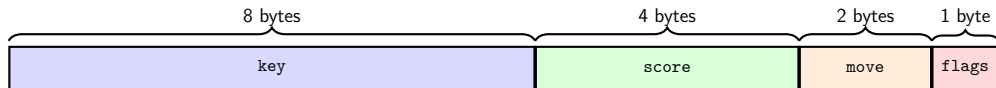
Hình 9: Sơ đồ phân bố bộ nhớ của một entry Bảng chuyển vị trong Lingine



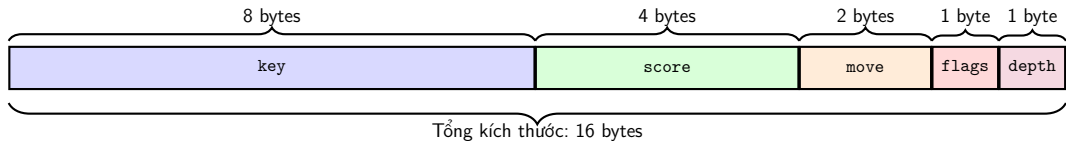
Hình 9: Sơ đồ phân bố bộ nhớ của một entry Bảng chuyển vị trong Lingine



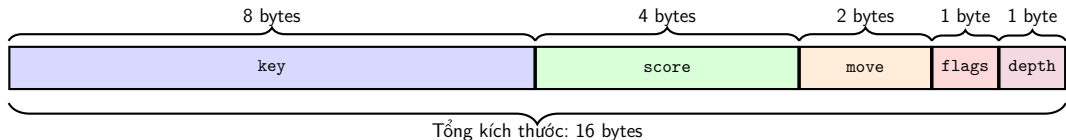
Hình 9: Sơ đồ phân bố bộ nhớ của một entry Bảng chuyển vị trong Lingine



Hình 9: Sơ đồ phân bố bộ nhớ của một entry Bảng chuyển vị trong Lingine



Hình 9: Sơ đồ phân bổ bộ nhớ của một entry Bảng chuyển vị trong Lingine



Hình 9: Sơ đồ phân bố bộ nhớ của một entry Bảng chuyển vị trong Lingine

Do tổng một entry là 16 bytes nên align tốt với cache, giúp tăng tốc độ truy vấn và tiết kiệm bộ nhớ.

Bài Toán

Biểu Diễn Trạng Thái

Sinh Nước Đi

Hàm Đánh Giá

Tìm Kiếm

Thực Nghiệm Và Demo

Kết luận



Gọi hàm `generate_moves(pos, type, &mut moves)`

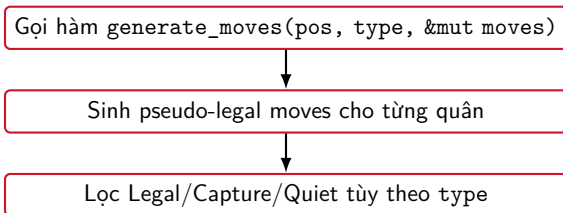
Hình 10: Pipeline sinh nước đi

Gọi hàm `generate_moves(pos, type, &mut moves)`

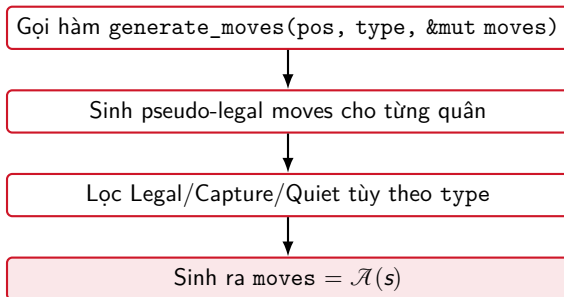


Sinh pseudo-legal moves cho từng quân

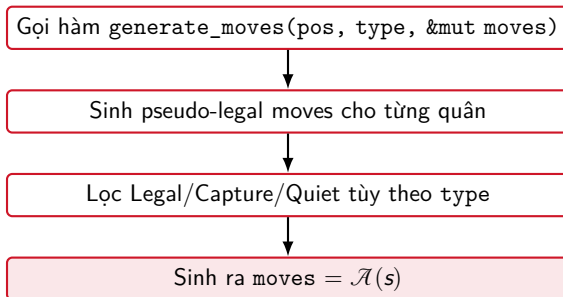
Hình 10: Pipeline sinh nước đi



Hình 10: Pipeline sinh nước đi



Hình 10: Pipeline sinh nước đi



Hình 10: Pipeline sinh nước đi

Biến `moves` có kiểu `MoveList` sử dụng `ArrayVec<Move, 128>` nằm hoàn toàn trên stack để tránh heap allocation.

Nhóm	Quân	Kỹ thuật
------	------	----------

Nhóm	Quân	Kỹ thuật
Không bị cản	Tướng, Sĩ, Tốt	Lookup table tĩnh, mask quân nhà.

Nhóm	Quân	Kỹ thuật
Không bị cản Bị cản	Tướng, Sĩ, Tốt Mã, Tượng	Lookup table tĩnh, mask quân nhà. Magic Bitboards cho chân/mắt.

Nhóm	Quân	Kỹ thuật
Không bị cản	Tướng, Sĩ, Tốt	Lookup table tĩnh, mask quân nhà.
Bị cản	Mã, Tượng	Magic Bitboards cho chân/mắt.
Trượt đường thẳng	Xe, Pháo	Rank/File table; Pháo tách quiet và capture.

Ở đây ta lấy ví dụ với quân Mã, quân Tượng làm tương tự.

- ◇ Mã có tối đa 8 nước.

Ở đây ta lấy ví dụ với quân Mã, quân Tượng làm tương tự.

- ◇ Mã có tối đa 8 nước.
- ◇ Nước nhảy phụ thuộc 4 ô chân Mã.

Ở đây ta lấy ví dụ với quân Mã, quân Tượng làm tương tự.

- ◇ Mã có tối đa 8 nước.
- ◇ Nước nhảy phụ thuộc 4 ô chân Mã.
- ◇ $\Rightarrow$ Có $2^4 = 16$ cách chặn khác nhau tại mỗi vị trí trên bàn cờ.

Ở đây ta lấy ví dụ với quân Mã, quân Tượng làm tương tự.

- ◇ Mã có tối đa 8 nước.
- ◇ Nước nhảy phụ thuộc 4 ô chân Mã.
- ◇ $\Rightarrow$ Có $2^4 = 16$ cách chặn khác nhau tại mỗi vị trí trên bàn cờ.
- ◇ $\Rightarrow$ Sinh ra toàn bộ các trường hợp có thể xảy ra.

Ở đây ta lấy ví dụ với quân Mã, quân Tượng làm tương tự.

- ◇ Mã có tối đa 8 nước.
- ◇ Nước nhảy phụ thuộc 4 ô chân Mã.
- ◇ $\Rightarrow$ Có $2^4 = 16$ cách chặn khác nhau tại mỗi vị trí trên bàn cờ.
- ◇ $\Rightarrow$ Sinh ra toàn bộ các trường hợp có thể xảy ra.

Vấn đề

Mỗi một trường hợp chặn được biểu diễn trên Bitboard có thể có giá trị rất lớn.

Ở đây ta lấy ví dụ với quân Mã, quân Tượng làm tương tự.

- ◇ Mã có tối đa 8 nước.
- ◇ Nước nhảy phụ thuộc 4 ô chân Mã.
- ◇ $\Rightarrow$ Có $2^4 = 16$ cách chặn khác nhau tại mỗi vị trí trên bàn cờ.
- ◇ $\Rightarrow$ Sinh ra toàn bộ các trường hợp có thể xảy ra.

Vấn đề

Mỗi một trường hợp chặn được biểu diễn trên Bitboard có thể có giá trị rất lớn.
 $\Rightarrow$ Cần có một cách ánh xạ chúng về 1 chỉ số từ 0 đến 15.

Occupied & Mask

Quân Mã (10 × 9)

.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	A	.	.	.	.	.
.	.	.	D	x	C	.	.	.	.
.	.	.	.	B	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.

Số ma thuật 128-bit

Chỉ số ánh xạ

4 bit cao nhất chứa [A B C D]

Occupied & Mask

Quân Mã (10 × 9)

.
.
.
.
. . . . A
. . . D x C . . .
. . . . B
.
.
.

Số ma thuật 128-bit

×

Chỉ số ánh xạ

4 bit cao nhất chứa [A B C D]

Occupied & Mask

Quân Mã (10×9)

.
.
.
.
. . . . A
. . . D x C . . .
. . . . B
.
.
.

Số ma thuật 128-bit

.
.
.
.
. . . một số. . .
. bit ma thuật. .
.
.
.
.

Chỉ số ánh xạ

4 bit cao nhất chứa [A B C D]

Occupied & Mask

Quân Mã (10 × 9)

```

. . . . .
. . . . .
. . . . .
. . . . .
. . . . A . . . .
. . . D x C . . .
. . . . B . . . .
. . . . .
. . . . .
. . . . .

```

Số ma thuật 128-bit

```

. . . . .
. . . . .
. . . . .
. . . . .
. . . một số. . .
. bit ma thuật. .
. . . . .
. . . . .
. . . . .
. . . . .

```

× =

Chỉ số ánh xạ

4 bit cao nhất chứa [A B C D]

Occupied & Mask

Quân Mã (10×9)

Số ma thuật 128-bit

```

. . . . .
. . . . .
. . . . .
. . . . .
. . . một số. . .
. bit ma thuật. . .
. . . . .
. . . . .
. . . . .
. . . . .

```

Chỉ số ánh xạ
4 bit cao nhất chứa $[A \ B \ C \ D]$

```
[A B C D] . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . bit rác . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
. . . . . . . .
```



Occupied & Mask
Quân Mã (10 × 9)

```

. . . . .
. . . . .
. . . . .
. . . . .
. . . . A . . . .
. . . D x C . . .
. . . . B . . . .
. . . . .
. . . . .
. . . . .

```

Số ma thuật 128-bit

```

. . . . .
. . . . .
. . . . .
. . . . .
. . . một số. . .
. bit ma thuật. .
. . . . .
. . . . .
. . . . .
. . . . .

```

×

=

Chỉ số ánh xạ

4 bit cao nhất chứa [A B C D]

```

[A B C D] . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .

```

[A B C D]

.

.

.

.

. . . bit rác . .

.

.

.

.

Khi này ta chỉ cần lấy số nhận được shift phải cho 124 thì sẽ ra được một số 4 bit.

Bài Toán

Biểu Diễn Trạng Thái

Sinh Nước Đi

Hàm Đánh Giá

Tìm Kiếm

Thực Nghiệm Và Demo

Kết luận



Search không thể duyệt tới kết thúc ván trong mọi lượt.

- ◇ Cây trò chơi bùng nổ theo cấp số nhân.

Search không thể duyệt tới kết thúc ván trong mọi lượt.

- ◇ Cây trò chơi bùng nổ theo cấp số nhân.
- ◇ Node lá cần được lượng hóa bằng điểm số.

Search không thể duyệt tới kết thúc ván trong mọi lượt.

- ◇ Cây trò chơi bùng nổ theo cấp số nhân.
- ◇ Node lá cần được lượng hóa bằng điểm số.
- ◇ Điểm dương nghiêng về Đỏ, điểm âm nghiêng về Đen.

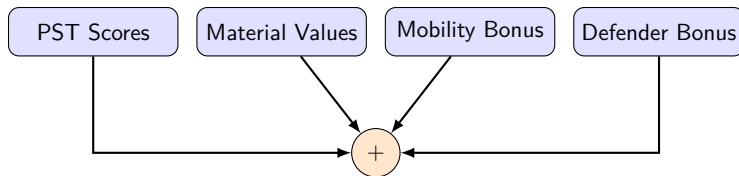
PST Scores

Material Values

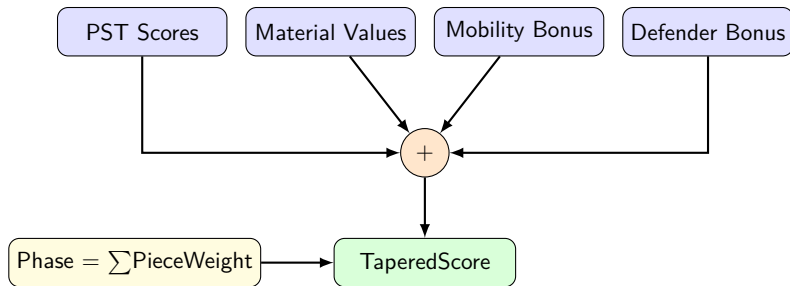
Mobility Bonus

Defender Bonus

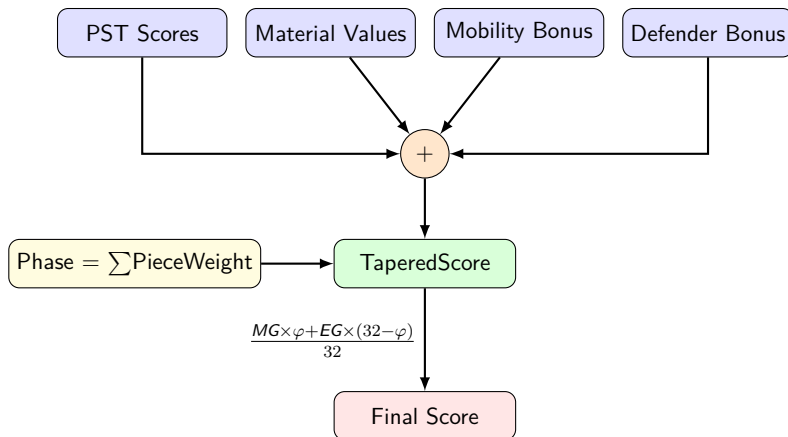
Hình 11: Sơ đồ pipeline hàm đánh giá thể cờ trong Lngine



Hình 11: Sơ đồ pipeline hàm đánh giá thể cờ trong Lingine



Hình 11: Sơ đồ pipeline hàm đánh giá thể cờ trong Lingine



Hình 11: Sơ đồ pipeline hàm đánh giá thể cờ trong Lingine

Mục tiêu: Thay thế thiết lập trọng số thủ công bằng thuật toán *Học máy Giám sát* (Hồi quy Logistic) trên hàng triệu thể cờ.

Mục tiêu: Thay thế thiết lập trọng số thủ công bằng thuật toán *Học máy Giám sát* (Hồi quy Logistic) trên hàng triệu thể cờ.

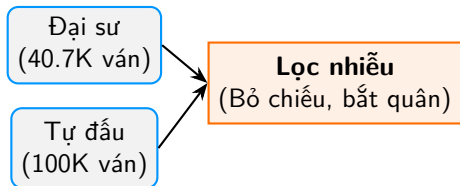
Quy trình Sinh & Tiền xử lý Dữ liệu

Đại sư
(40.7K ván)

Tự đấu
(100K ván)

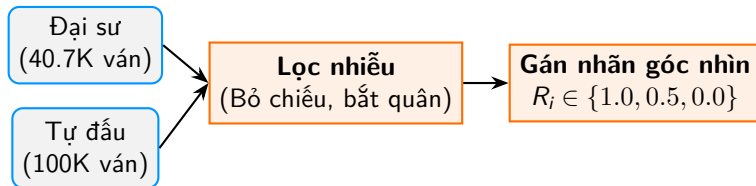
Mục tiêu: Thay thế thiết lập trọng số thủ công bằng thuật toán *Học máy Giám sát* (Hồi quy Logistic) trên hàng triệu thể cờ.

Quy trình Sinh & Tiền xử lý Dữ liệu



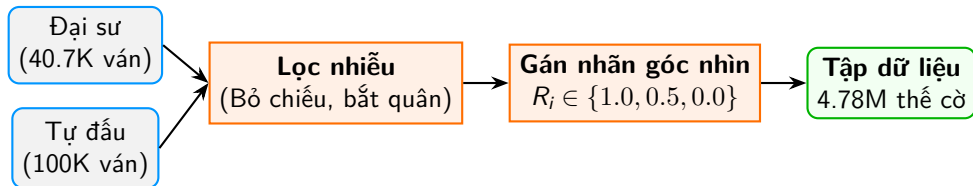
Mục tiêu: Thay thế thiết lập trọng số thủ công bằng thuật toán *Học máy Giám sát* (Hồi quy Logistic) trên hàng triệu thể cờ.

Quy trình Sinh & Tiền xử lý Dữ liệu



Mục tiêu: Thay thế thiết lập trọng số thủ công bằng thuật toán *Học máy Giám sát* (Hồi quy Logistic) trên hàng triệu thể cờ.

Quy trình Sinh & Tiền xử lý Dữ liệu



Mục tiêu: Tối thiểu hóa sai số toàn phương trung bình (**MSE**) để tìm bộ tham số $\vec{\theta}$:

$$\text{MSE}(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \left(R_i - \sigma \left(E(S_i, \vec{\theta}) \right) \right)^2 \quad \text{với} \quad \sigma(x) = \frac{1}{1 + 10^{-\frac{K \cdot x}{400}}}$$

Trong đó:

Mục tiêu: Tối thiểu hóa sai số toàn phương trung bình (**MSE**) để tìm bộ tham số $\vec{\theta}$:

$$\text{MSE}(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \left(R_i - \sigma \left(E(S_i, \vec{\theta}) \right) \right)^2 \quad \text{với} \quad \sigma(x) = \frac{1}{1 + 10^{-\frac{K \cdot x}{400}}}$$

Trong đó:

◇ $R_i \in \{1.0, 0.5, 0.0\}$ là kết quả thực tế của ván đấu chứa thẻ cờ thứ i .

Mục tiêu: Tối thiểu hóa sai số toàn phương trung bình (**MSE**) để tìm bộ tham số $\vec{\theta}$:

$$\text{MSE}(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \left(R_i - \sigma \left(E(S_i, \vec{\theta}) \right) \right)^2 \quad \text{với} \quad \sigma(x) = \frac{1}{1 + 10^{-\frac{K \cdot x}{400}}}$$

Trong đó:

- ◇ $R_i \in \{1.0, 0.5, 0.0\}$ là kết quả thực tế của ván đấu chứa thể cờ thứ i .
- ◇ $E(S_i, \vec{\theta})$ là hàm đánh giá tĩnh thể cờ S_i bằng bộ tham số $\vec{\theta}$.

Mục tiêu: Tối thiểu hóa sai số toàn phương trung bình (**MSE**) để tìm bộ tham số $\vec{\theta}$:

$$\text{MSE}(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \left(R_i - \sigma \left(E(S_i, \vec{\theta}) \right) \right)^2 \quad \text{với} \quad \sigma(x) = \frac{1}{1 + 10^{-\frac{K \cdot x}{400}}}$$

Trong đó:

- ◇ $R_i \in \{1.0, 0.5, 0.0\}$ là kết quả thực tế của ván đấu chứa thể cờ thứ i .
- ◇ $E(S_i, \vec{\theta})$ là hàm đánh giá tĩnh thể cờ S_i bằng bộ tham số $\vec{\theta}$.
- ◇ $\sigma(x)$ là hàm kích hoạt Sigmoid có nhiệm vụ ánh xạ điểm số cờ sang $[0, 1]$ đại diện cho cơ hội chiến thắng dự kiến.

Mục tiêu: Tối thiểu hóa sai số toàn phương trung bình (**MSE**) để tìm bộ tham số $\vec{\theta}$:

$$\text{MSE}(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \left(R_i - \sigma \left(E(S_i, \vec{\theta}) \right) \right)^2 \quad \text{với} \quad \sigma(x) = \frac{1}{1 + 10^{-\frac{K \cdot x}{400}}}$$

Trong đó:

- ◇ $R_i \in \{1.0, 0.5, 0.0\}$ là kết quả thực tế của ván đấu chứa thể cờ thứ i .
- ◇ $E(S_i, \vec{\theta})$ là hàm đánh giá tĩnh thể cờ S_i bằng bộ tham số $\vec{\theta}$.
- ◇ $\sigma(x)$ là hàm kích hoạt Sigmoid có nhiệm vụ ánh xạ điểm số cờ sang $[0, 1]$ đại diện cho cơ hội chiến thắng dự kiến.

Giải pháp

Sử dụng các thuật toán tối ưu để tìm lời giải cho bài toán trên, Lingine đã sử dụng Adam Optimizer.

Giai đoạn	Hệ số K	MSE
Khởi tạo	0.233	0.0620
Hội tụ	0.549	0.0550

Giai đoạn	Hệ số K	MSE
Khởi tạo	0.233	0.0620
Hội tụ	0.549	0.0550

Đánh giá

Hàm mất mát MSE giảm
> **11%**. Bộ tham số tự
hiệu chỉnh chính xác theo
dữ liệu lớn.

Giai đoạn	Hệ số K	MSE
Khởi tạo	0.233	0.0620
Hội tụ	0.549	0.0550

Đánh giá

Hàm mất mát MSE giảm **>11%**. Bộ tham số tự hiệu chỉnh chính xác theo dữ liệu lớn.

Chỉ số	Lingine 1.8.0	Lingine 1.7.0
W/D/L	719 / 156 / 125	125 / 156 / 719
Win Rate	79.7%	20.3%
Elo Diff	+237.6	-237.6
LOS	100.0%	—

Bảng 1: Lingine 1.8.0 vs Lingine 1.7.0

Bài Toán

Biểu Diễn Trạng Thái

Sinh Nước Đi

Hàm Đánh Giá

Tìm Kiếm

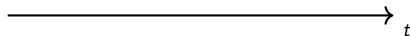
Thực Nghiệm Và Demo

Kết luận

Phân bổ ngân sách thông minh để tối ưu độ sâu tính toán, đồng thời đảm bảo không bị xử thua do cạn giờ (luôn trừ hao $\text{TimeOverhead} = 15\text{ms}$).

Cơ chế phân bổ:

Gọi $\text{Total} = \text{TimeLeft} + \text{Increment}$

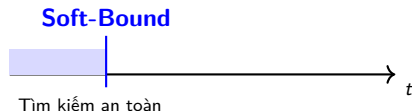


Phân bổ ngân sách thông minh để tối ưu độ sâu tính toán, đồng thời đảm bảo không bị xử thua do cạn giờ (luôn trừ hao TimeOverhead = 15ms).

Cơ chế phân bổ:

Gọi $Total = TimeLeft + Increment$

- ◇ **Soft-Bound** ($\approx \frac{Total}{40}$):
 - Tích hợp ở vòng lặp *Iterative Deepening*.
 - Chốt kết quả ở độ sâu d và ngắt lặp để tránh rủi ro.

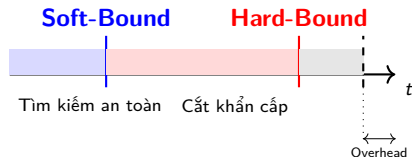


Phân bổ ngân sách thông minh để tối ưu độ sâu tính toán, đồng thời đảm bảo không bị xử thua do cạn giờ (luôn trừ hao TimeOverhead = 15ms).

Cơ chế phân bổ:

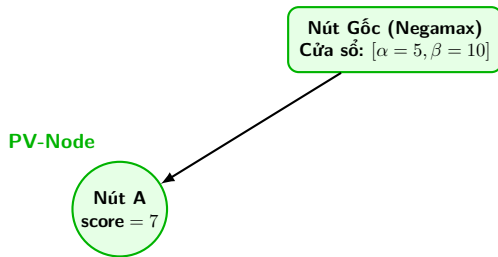
Gọi $Total = TimeLeft + Increment$

- ◇ **Soft-Bound** ($\approx \frac{Total}{40}$):
 - Tích hợp ở vòng lặp *Iterative Deepening*.
 - Chốt kết quả ở độ sâu d và ngắt lặp để tránh rủi ro.
- ◇ **Hard-Bound** ($\approx \frac{Total}{8}$):
 - Công tắc khẩn cấp trong nhánh *Negamax*.

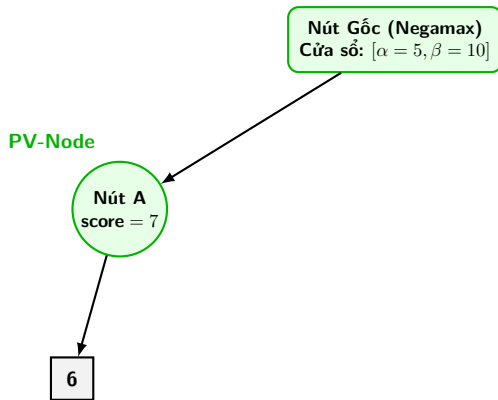


Nút Gốc (Negamax)
Cửa sổ: $[\alpha = 5, \beta = 10]$

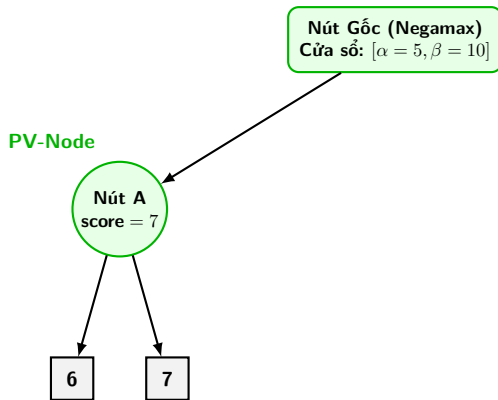
Hình 12: Minh họa hành vi thuật toán Alpha-Beta Negamax



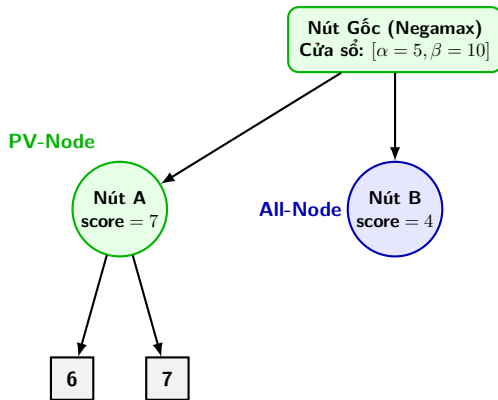
Hình 12: Minh họa hành vi thuật toán Alpha-Beta Negamax



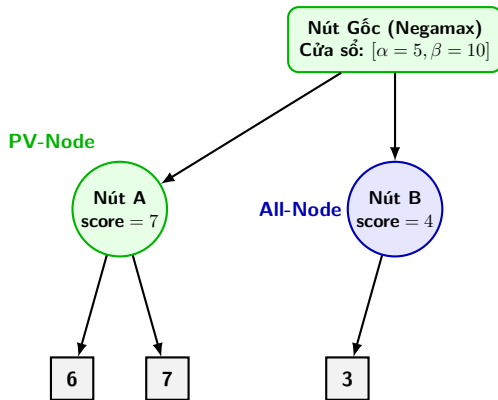
Hình 12: Minh họa hành vi thuật toán Alpha-Beta Negamax



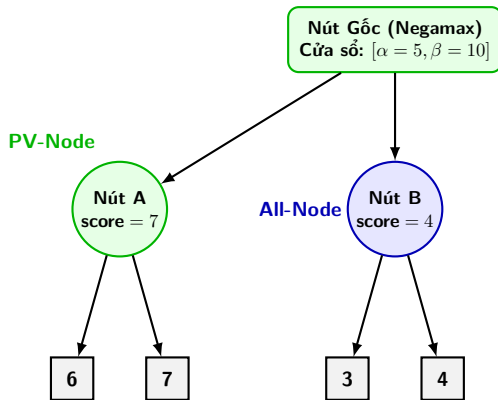
Hình 12: Minh họa hành vi thuật toán Alpha-Beta Negamax



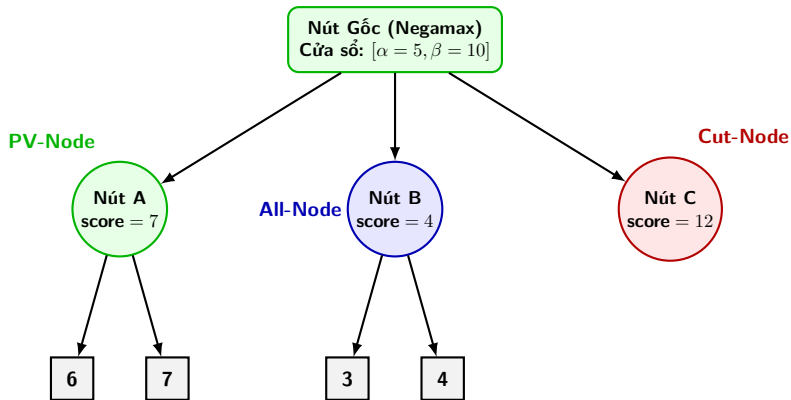
Hình 12: Minh họa hành vi thuật toán Alpha-Beta Negamax



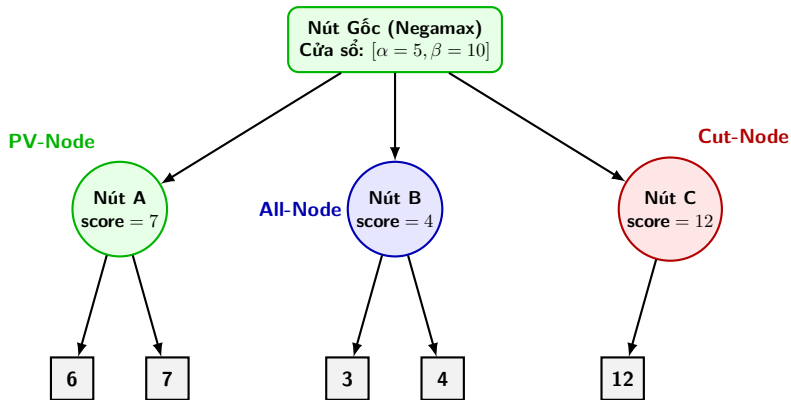
Hình 12: Minh họa hành vi thuật toán Alpha-Beta Negamax



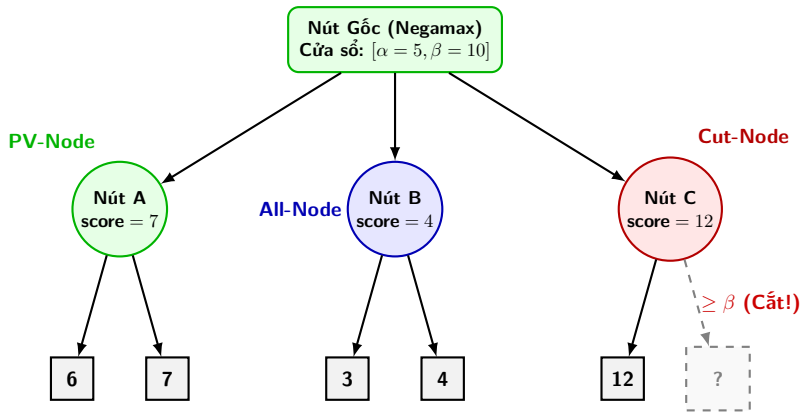
Hình 12: Minh họa hành vi thuật toán Alpha-Beta Negamax



Hình 12: Minh họa hành vi thuật toán Alpha-Beta Negamax



Hình 12: Minh họa hành vi thuật toán Alpha-Beta Negamax



Hình 12: Minh họa hành vi thuật toán Alpha-Beta Negamax

Mức độ ưu tiên



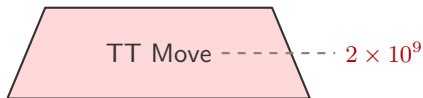
Hình 13: Tháp phân cấp thứ tự duyệt nước đi trong Lengine.

Mức độ ưu tiên



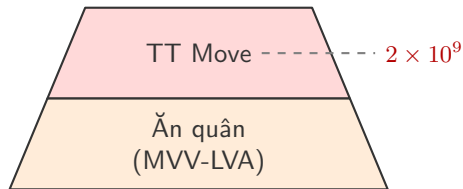
Hình 13: Tháp phân cấp thứ tự duyệt nước đi trong Lingine.

Mức độ ưu tiên



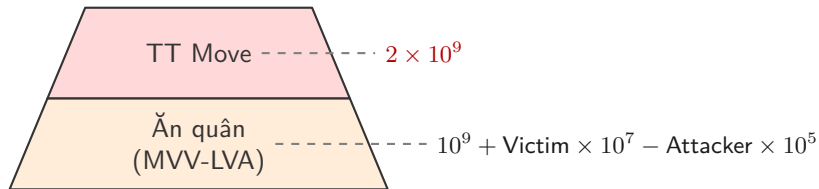
Hình 13: Tháp phân cấp thứ tự duyệt nước đi trong Lingine.

Mức độ ưu tiên



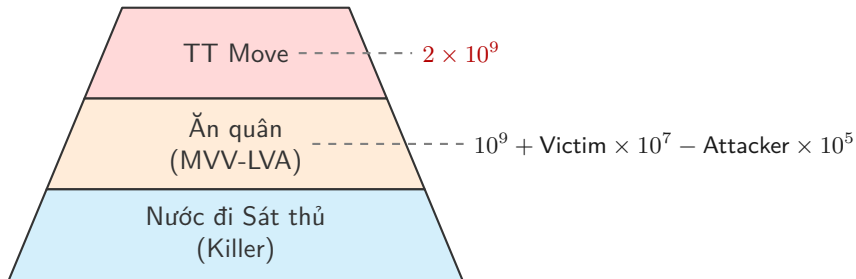
Hình 13: Tháp phân cấp thứ tự duyệt nước đi trong Lengine.

Mức độ ưu tiên



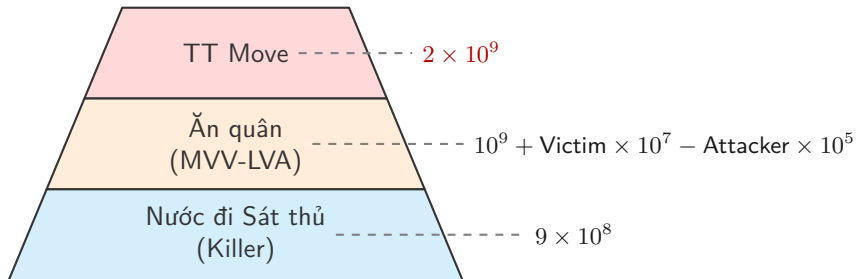
Hình 13: Tháp phân cấp thứ tự duyệt nước đi trong Lengine.

Mức độ ưu tiên



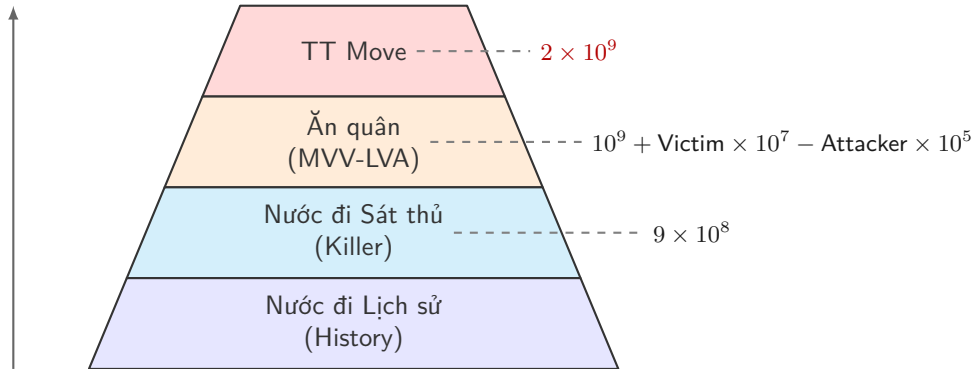
Hình 13: Tháp phân cấp thứ tự duyệt nước đi trong Lingine.

Mức độ ưu tiên



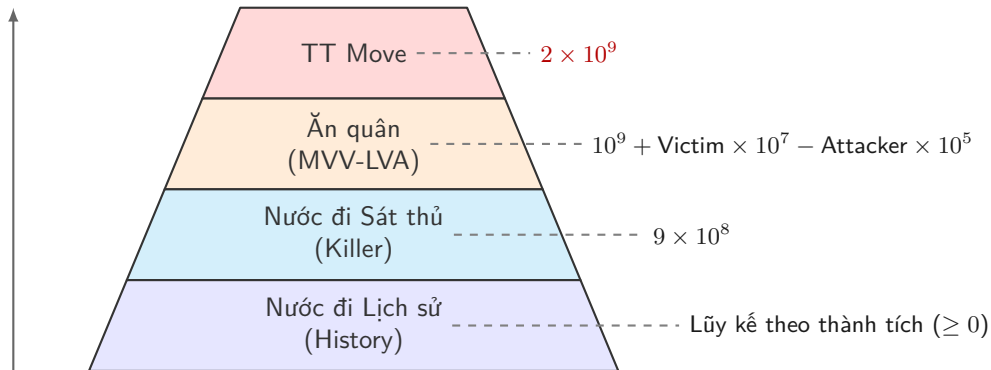
Hình 13: Tháp phân cấp thứ tự duyệt nước đi trong Lingine.

Mức độ ưu tiên



Hình 13: Tháp phân cấp thứ tự duyệt nước đi trong Lingine.

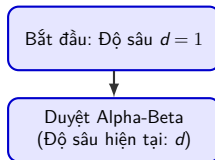
Mức độ ưu tiên



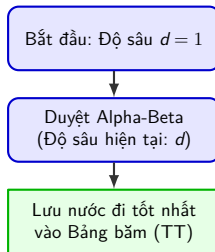
Hình 13: Tháp phân cấp thứ tự duyệt nước đi trong Lingine.

Bắt đầu: Độ sâu $d = 1$

Hình 14: Lưu đồ thuật toán Tìm kiếm sâu dần (Iterative Deepening)

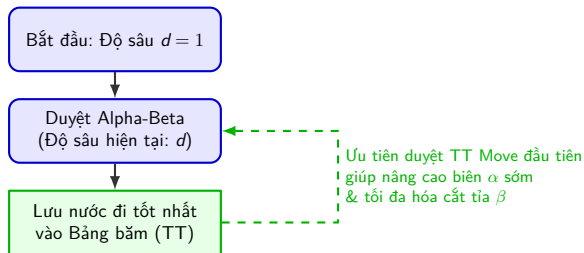


Hình 14: Lưu đồ thuật toán Tìm kiếm sâu dần (Iterative Deepening)



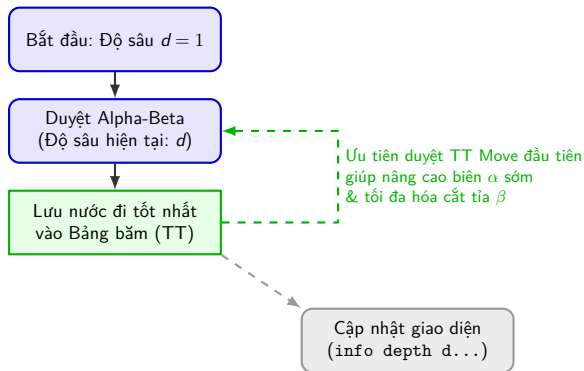
Hình 14: Lưu đồ thuật toán Tìm kiếm sâu dần (Iterative Deepening)

Tìm kiếm sâu dần (Iterative Deepening)



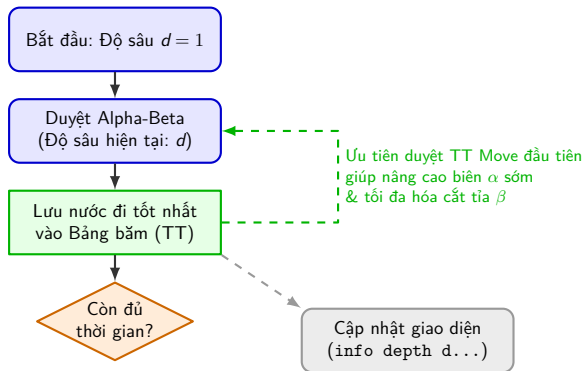
Hình 14: Lưu đồ thuật toán Tìm kiếm sâu dần (Iterative Deepening)

Tìm kiếm sâu dần (Iterative Deepening)



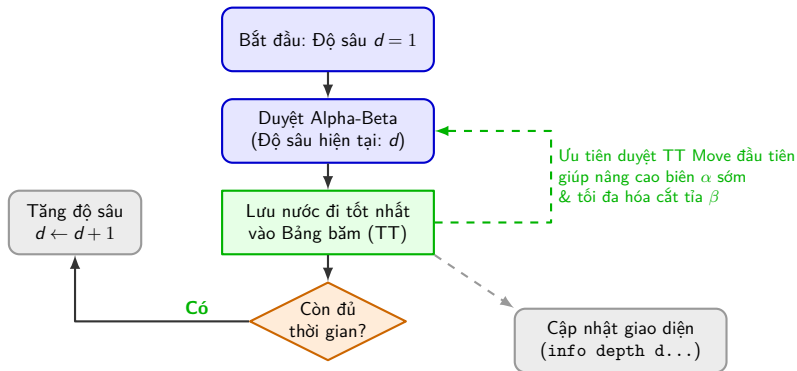
Hình 14: Lưu đồ thuật toán Tìm kiếm sâu dần (Iterative Deepening)

Tìm kiếm sâu dần (Iterative Deepening)



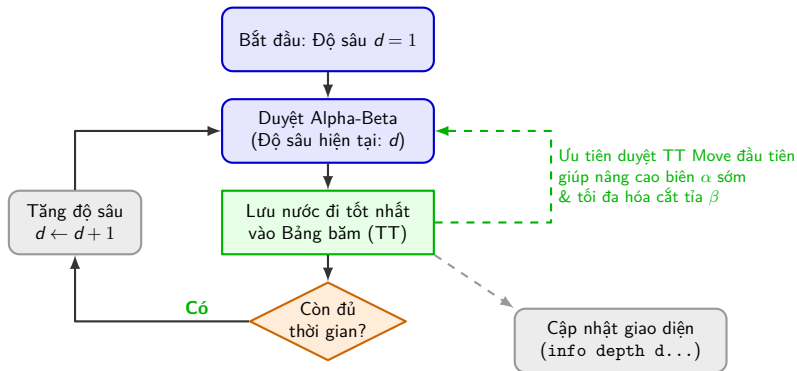
Hình 14: Lưu đồ thuật toán Tìm kiếm sâu dần (Iterative Deepening)

Tìm kiếm sâu dần (Iterative Deepening)



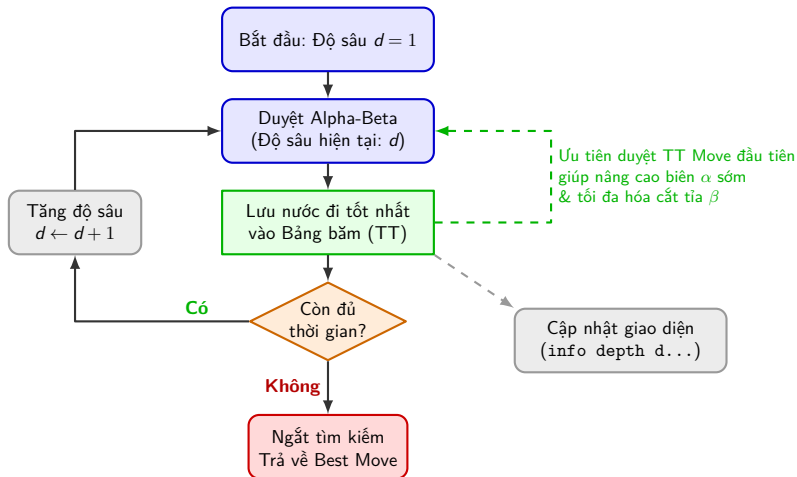
Hình 14: Lưu đồ thuật toán Tìm kiếm sâu dần (Iterative Deepening)

Tìm kiếm sâu dần (Iterative Deepening)



Hình 14: Lưu đồ thuật toán Tìm kiếm sâu dần (Iterative Deepening)

Tìm kiếm sâu dần (Iterative Deepening)

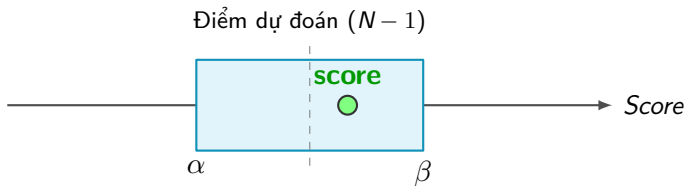


Hình 14: Lưu đồ thuật toán Tìm kiếm sâu dần (Iterative Deepening)

- ◇ **Ý tưởng cốt lõi:** Nhờ kết quả từ độ sâu $N - 1$, thuật toán không dùng cửa sổ vô cực $[-\infty, +\infty]$ mà khởi tạo một cửa sổ hẹp $[\alpha, \beta]$ để cắt tỉa nhánh mạnh hơn.

- ◇ **Ý tưởng cốt lõi:** Nhờ kết quả từ độ sâu $N - 1$, thuật toán không dùng cửa sổ vô cực $[-\infty, +\infty]$ mà khởi tạo một cửa sổ hẹp $[\alpha, \beta]$ để cắt tỉa nhánh mạnh hơn.
- ◇ **1. Kích bản Thành công** ($\alpha < \text{score} < \beta$):
 - Điểm số lọt vào cửa sổ $\rightarrow$ Tìm kiếm hoàn tất, trả về điểm số.

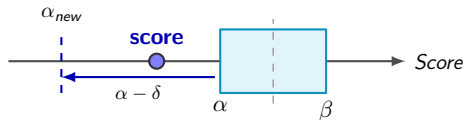
- ◇ **Ý tưởng cốt lõi:** Nhờ kết quả từ độ sâu $N - 1$, thuật toán không dùng cửa sổ vô cực $[-\infty, +\infty]$ mà khởi tạo một cửa sổ hẹp $[\alpha, \beta]$ để cắt tỉa nhánh mạnh hơn.
- ◇ **1. Kích bản Thành công** ($\alpha < \text{score} < \beta$):
 - Điểm số lọt vào cửa sổ $\rightarrow$ Tìm kiếm hoàn tất, trả về điểm số.



◇ 2. Fail-low (score $\leq \alpha$):

- Thể trận rủi ro hơn dự kiến.
- Mở rộng đáy: $\alpha_{new} = \alpha - \delta$.
- Duyệt lại: **Giữ nguyên độ sâu.**

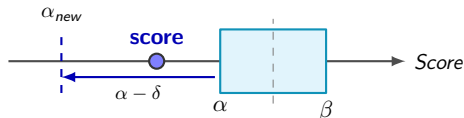
Kịch bản Fail-low



◇ 2. Fail-low (score $\leq \alpha$):

- Thể trận rủi ro hơn dự kiến.
- Mở rộng đáy: $\alpha_{new} = \alpha - \delta$.
- Duyệt lại: **Giữ nguyên độ sâu**.

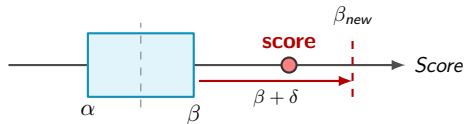
Kịch bản Fail-low



◇ 3. Fail-high (score $\geq \beta$):

- Nước đi áp đảo (*Fail-high Reductions*).
- Mở rộng đỉnh: $\beta_{new} = \beta + \delta$.
- Tăng đếm fail_high.
- Duyệt lại: **Giảm độ sâu** xuống $\max(1, \text{depth} - \text{fail_high})$.

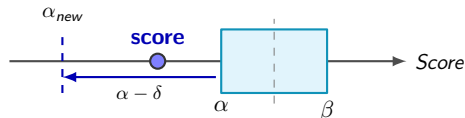
Kịch bản Fail-high



◇ 2. Fail-low (score $\leq \alpha$):

- Thể trận rủi ro hơn dự kiến.
- Mở rộng đáy: $\alpha_{new} = \alpha - \delta$.
- Duyệt lại: **Giữ nguyên độ sâu**.

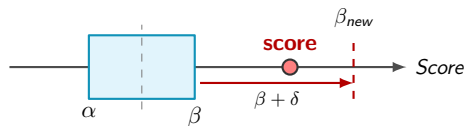
Kịch bản Fail-low



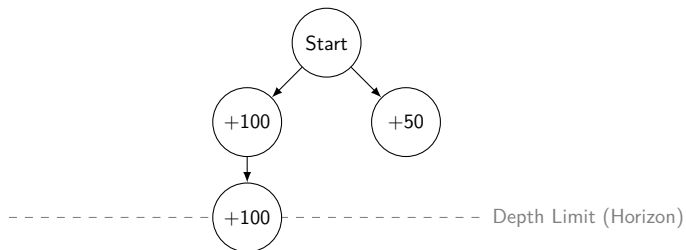
◇ 3. Fail-high (score $\geq \beta$):

- Nước đi áp đảo (*Fail-high Reductions*).
- Mở rộng đỉnh: $\beta_{new} = \beta + \delta$.
- Tăng đếm fail_high.
- Duyệt lại: **Giảm độ sâu** xuống $\max(1, \text{depth} - \text{fail_high})$.

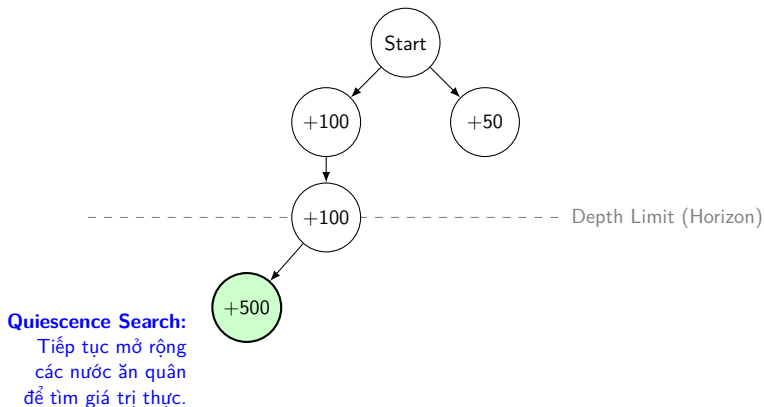
Kịch bản Fail-high



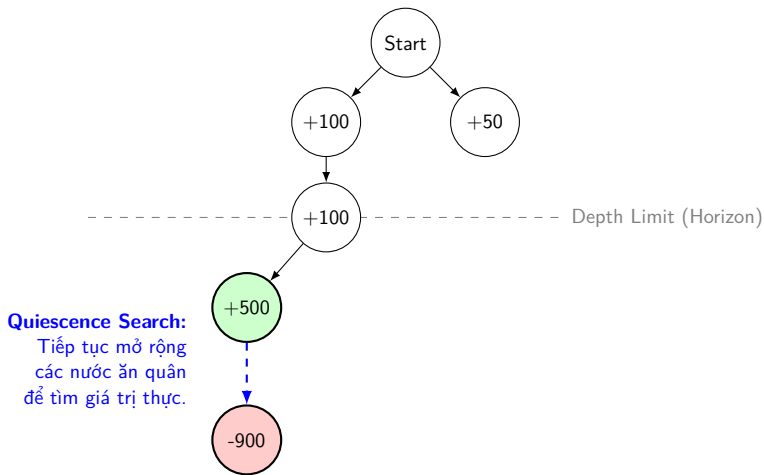
- ◇ **Lưu ý:** $\delta = \delta \times 2$ sau mỗi lần trượt để mở rộng nhanh hơn.



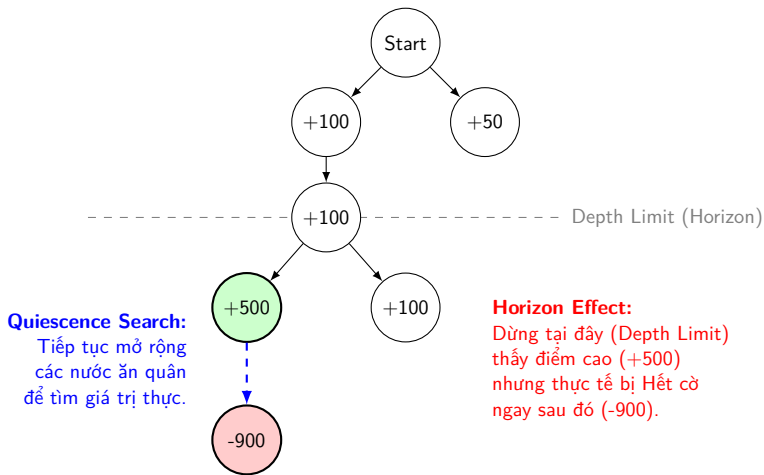
Hình 15: Minh họa Horizon Effect và Tìm kiếm tĩnh



Hình 15: Minh họa Horizon Effect và Tìm kiếm tĩnh

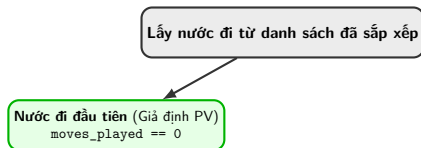


Hình 15: Minh họa Horizon Effect và Tìm kiếm tĩnh

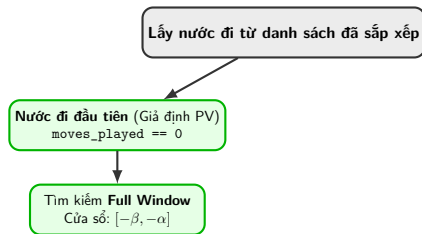


Hình 15: Minh họa Horizon Effect và Tìm kiếm tĩnh

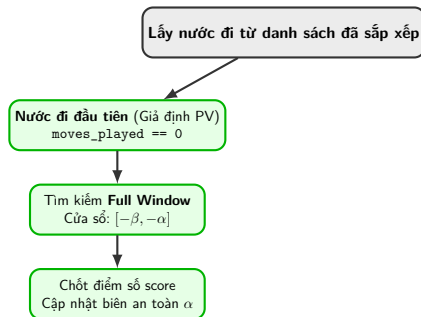
Hình 16: Lưu đồ giải thuật Principal Variation Search.



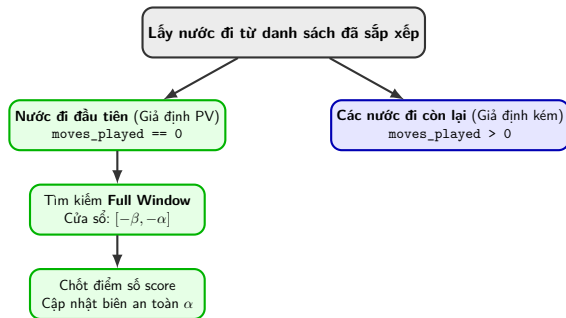
Hình 16: Lưu đồ giải thuật Principal Variation Search.



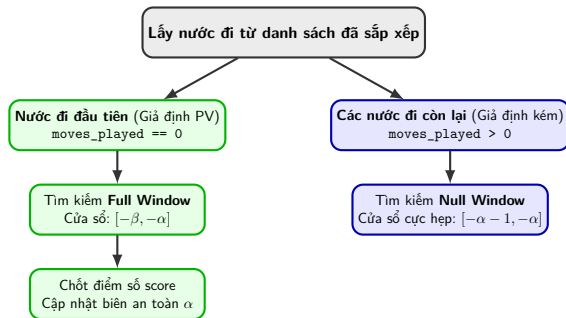
Hình 16: Lưu đồ giải thuật Principal Variation Search.



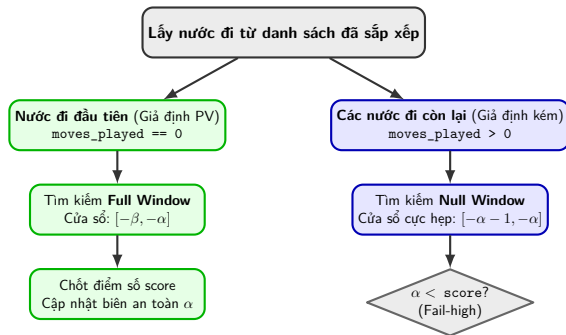
Hình 16: Lưu đồ giải thuật Principal Variation Search.



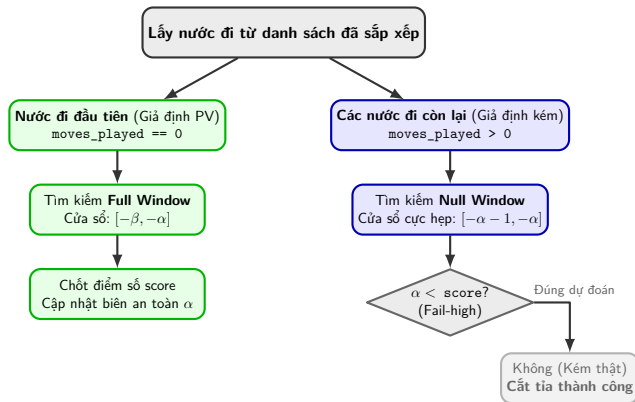
Hình 16: Lưu đồ giải thuật Principal Variation Search.



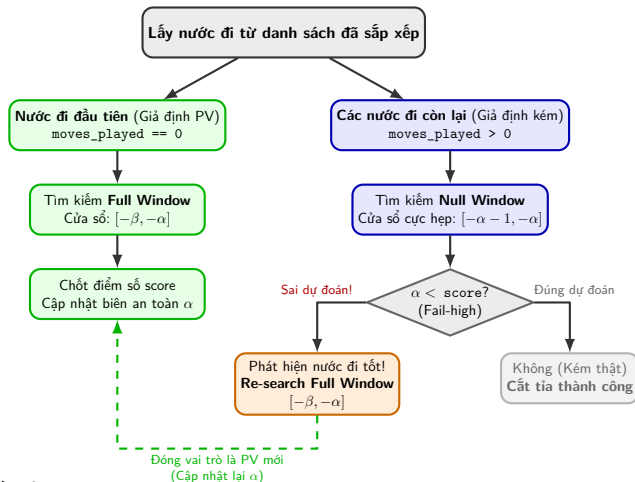
Hình 16: Lưu đồ giải thuật Principal Variation Search.



Hình 16: Lưu đồ giải thuật Principal Variation Search.



Hình 16: Lưu đồ giải thuật Principal Variation Search.



Mục tiêu: Bù đắp độ sâu tại các thời điểm nhảy cảm để tránh *Hiệu ứng chân trời* (Horizon Effect).

Mục tiêu: Bù đắp độ sâu tại các thời điểm nhảy cảm để tránh *Hiệu ứng chân trời* (Horizon Effect).

◇ **1. Mở rộng khi bị chiếu (Check Extensions)**

- *Bối cảnh:* Mọi nước đi đều bắt buộc, dễ dẫn đến biến động vật chất lớn.
- *Xử lý:* Nếu Tướng đang bị chiếu $\Rightarrow$ Tăng độ sâu ($d \leftarrow d + 1$) để phòng thủ hoặc tìm sát cục.

Mục tiêu: Bù đắp độ sâu tại các thời điểm nhảy cảm để tránh *Hiệu ứng chân trời* (Horizon Effect).

◇ **1. Mở rộng khi bị chiếu (Check Extensions)**

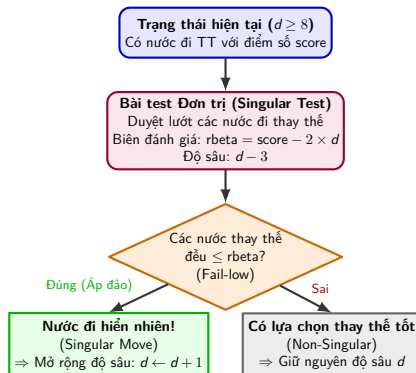
- *Bối cảnh:* Mọi nước đi đều bắt buộc, dễ dẫn đến biến động vật chất lớn.
- *Xử lý:* Nếu Tướng đang bị chiếu $\Rightarrow$ Tăng độ sâu ($d \leftarrow d + 1$) để phòng thủ hoặc tìm sát cục.

◇ **2. Mở rộng nước đi duy nhất (One-Reply Extensions)**

- *Bối cảnh:* Nút mạng chỉ có đúng 1 nước đi hợp lệ.
- *Đặc điểm:* Đào sâu thêm 1 ply gần như **miễn phí** tài nguyên (hệ số rẽ nhánh = 1).
- *Xử lý:* Bổ sung tầm nhìn chiến thuật bằng cách tăng độ sâu ($d \leftarrow d + 1$).

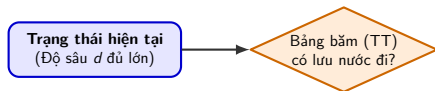
- ◇ **Ý tưởng:** Một nước đi tỏ ra quá áp đảo so với phần còn lại $\Rightarrow$ Nước đi "hiển nhiên". Cần đào sâu để đề phòng bẫy chiến thuật ẩn.

- ◇ **Ý tưởng:** Một nước đi tỏ ra quá áp đảo so với phần còn lại $\Rightarrow$ Nước đi "hiển nhiên". Cần đào sâu để đề phòng bất chiến thuật ẩn.



Hình 17: Cơ chế Mở rộng đơn trị (Singular Extensions).

- ◇ **Vấn đề:** Ở độ sâu d lớn, nếu Bảng băm (TT) không có dữ liệu $\Rightarrow$ Rủi ro cao đây là thể cờ kém từng bị cắt tỉa trong quá khứ.



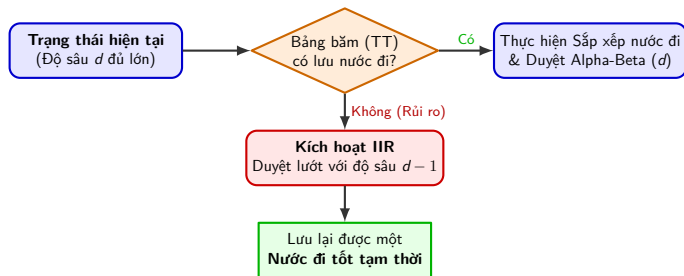
Hình 18: Cơ chế Giảm độ sâu lặp nội bộ.

- ◇ **Vấn đề:** Ở độ sâu d lớn, nếu Bảng băm (TT) không có dữ liệu $\Rightarrow$ Rủi ro cao đây là thế cờ kém từng bị cắt tỉa trong quá khứ.



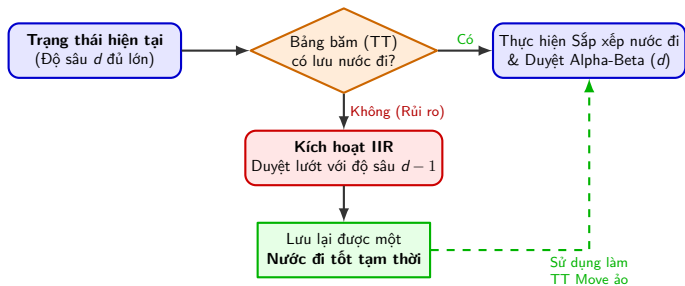
Hình 18: Cơ chế Giảm độ sâu lặp nội bộ.

- ◇ **Vấn đề:** Ở độ sâu d lớn, nếu Bảng băm (TT) không có dữ liệu $\Rightarrow$ Rủi ro cao đây là thể cờ kém từng bị cắt tỉa trong quá khứ.
- ◇ **Giải pháp (IIR):** Kích hoạt quét nhanh ở độ sâu $d - 1$ để tìm một nước đi "tạm ổn", sau đó dùng nó làm mỏ neo định hướng để duyệt tiếp.



Hình 18: Cơ chế Giảm độ sâu lặp nội bộ.

- ◇ **Vấn đề:** Ở độ sâu d lớn, nếu Bảng băm (TT) không có dữ liệu $\Rightarrow$ Rủi ro cao đây là thế cờ kém từng bị cắt tỉa trong quá khứ.
- ◇ **Giải pháp (IIR):** Kích hoạt quét nhanh ở độ sâu $d - 1$ để tìm một nước đi "tạm ổn", sau đó dùng nó làm mỏ neo định hướng để duyệt tiếp.



Hình 18: Cơ chế Giảm độ sâu lặp nội bộ.

- ◇ **Ý tưởng:** "Nếu ta **bỏ qua một lượt đi** mà thế trận vẫn đủ mạnh để nghiền nát đối thủ ($\text{score} \geq \beta$), thì không cần tốn thời gian tính toán các nước đi hợp lệ nữa."

Hình 19: Minh họa cây tìm kiếm của Null Move Pruning (NMP).

- ◇ **Ý tưởng:** "Nếu ta **bỏ qua một lượt đi** mà thế trận vẫn đủ mạnh để nghiền nát đối thủ ($\text{score} \geq \beta$), thì không cần tốn thời gian tính toán các nước đi hợp lệ nữa."

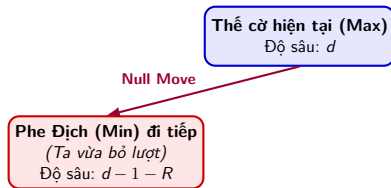
Hình 19: Minh họa cây tìm kiếm của Null Move Pruning (NMP).

- ◇ **Ý tưởng:** "Nếu ta **bỏ qua một lượt đi** mà thế trận vẫn đủ mạnh để nghiền nát đối thủ ($\text{score} \geq \beta$), thì không cần tốn thời gian tính toán các nước đi hợp lệ nữa."

Thế cờ hiện tại (Max)
Độ sâu: d

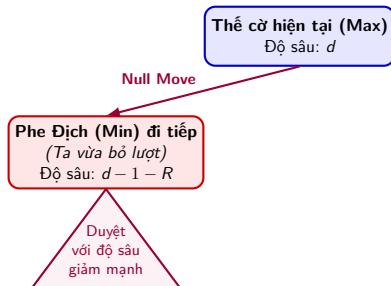
Hình 19: Minh họa cây tìm kiếm của Null Move Pruning (NMP).

- ◇ **Ý tưởng:** "Nếu ta **bỏ qua một lượt đi** mà thế trận vẫn đủ mạnh để nghiền nát đối thủ ($\text{score} \geq \beta$), thì không cần tốn thời gian tính toán các nước đi hợp lệ nữa."



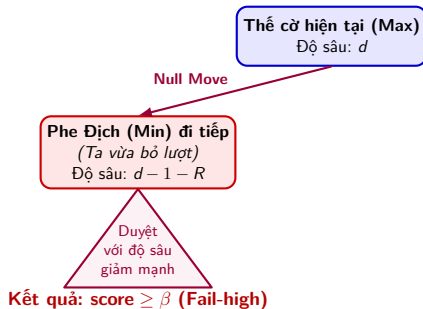
Hình 19: Minh họa cây tìm kiếm của Null Move Pruning (NMP).

- ◇ **Ý tưởng:** "Nếu ta **bỏ qua một lượt đi** mà thế trận vẫn đủ mạnh để nghiền nát đối thủ ($\text{score} \geq \beta$), thì không cần tốn thời gian tính toán các nước đi hợp lệ nữa."



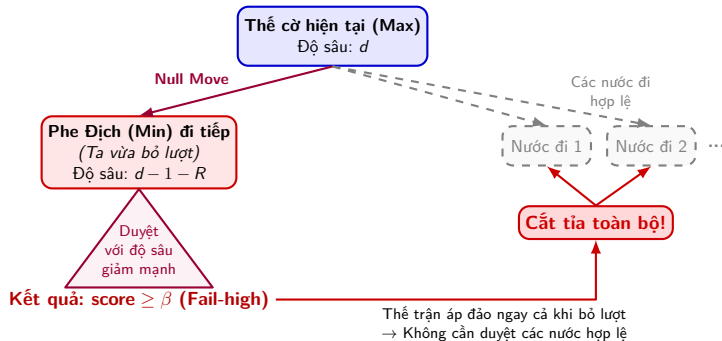
Hình 19: Minh họa cây tìm kiếm của Null Move Pruning (NMP).

- ◇ **Ý tưởng:** "Nếu ta **bỏ qua một lượt đi** mà thế trận vẫn đủ mạnh để nghiền nát đối thủ ($\text{score} \geq \beta$), thì không cần tốn thời gian tính toán các nước đi hợp lệ nữa."



Hình 19: Minh họa cây tìm kiếm của Null Move Pruning (NMP).

- ◇ **Ý tưởng:** "Nếu ta **bỏ qua một lượt đi** mà thế trận vẫn đủ mạnh để nghiền nát đối thủ ($\text{score} \geq \beta$), thì không cần tốn thời gian tính toán các nước đi hợp lệ nữa."

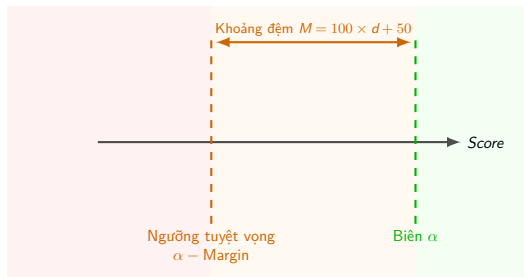


Hình 19: Minh họa cây tìm kiếm của Null Move Pruning (NMP).

- ◇ **Ý tưởng:** Khi thể cờ quá lép ($\text{Eval} \ll \alpha$) và không bị chiếu, các nước đi bình thường (*quiet moves*) không thể tạo ra phép màu lật ngược tình thế $\rightarrow$ Bỏ qua.

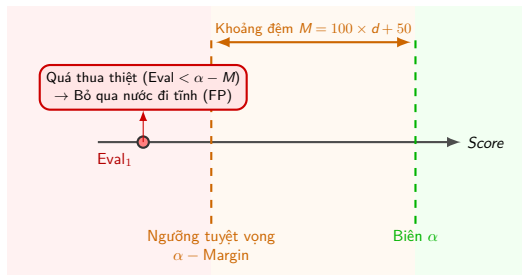
Hình 20: Minh họa Cắt tỉa vô ích.

- ◇ **Ý tưởng:** Khi thể cờ quá lép ($\text{Eval} \ll \alpha$) và không bị chiếu, các nước đi bình thường (*quiet moves*) không thể tạo ra phép màu lật ngược tình thế $\rightarrow$ Bỏ qua.



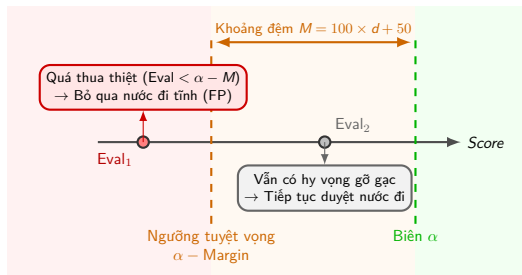
Hình 20: Minh họa Cắt tỉa vô ích.

- ◇ **Ý tưởng:** Khi thể cờ quá lép ($\text{Eval} \ll \alpha$) và không bị chiếu, các nước đi bình thường (*quiet moves*) không thể tạo ra phép màu lật ngược tình thế $\rightarrow$ Bỏ qua.



Hình 20: Minh họa Cắt tỉa vô ích.

- ◇ **Ý tưởng:** Khi thế cờ quá lép ($Eval \ll \alpha$) và không bị chiếu, các nước đi bình thường (*quiet moves*) không thể tạo ra phép màu lật ngược tình thế $\rightarrow$ Bỏ qua.

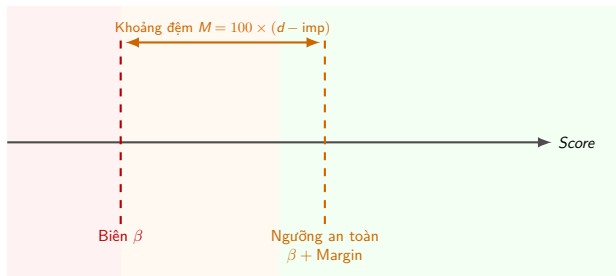


Hình 20: Minh họa Cắt tỉa vô ích.

- ◇ **Ý tưởng:** Đánh giá tĩnh (*static eval*) vượt xa biên β một khoảng an toàn lớn $\rightarrow$ Thế trận quá tốt, có thể cắt tỉa nhánh ngay mà không cần tốn công thử nước đi.

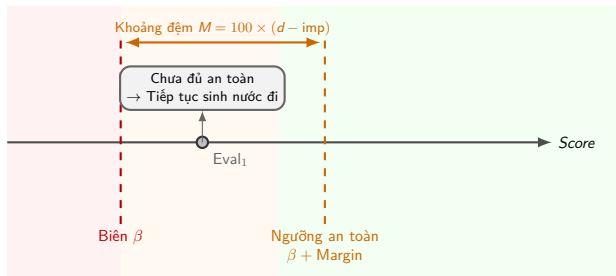
Hình 21: Minh họa Cắt tỉa vô ích ngược.

- ◇ **Ý tưởng:** Đánh giá tĩnh (*static eval*) vượt xa biên β một khoảng an toàn lớn $\rightarrow$ Thế trận quá tốt, có thể cắt tỉa nhánh ngay mà không cần tốn công thử nước đi.



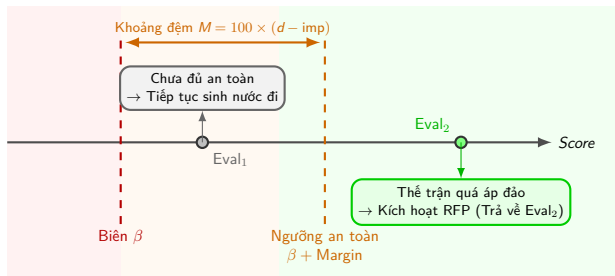
Hình 21: Minh họa Cắt tỉa vô ích ngược.

- ◇ **Ý tưởng:** Đánh giá tĩnh (*static eval*) vượt xa biên β một khoảng an toàn lớn $\rightarrow$ Thế trận quá tốt, có thể cắt tỉa nhánh ngay mà không cần tốn công thử nước đi.



Hình 21: Minh họa Cắt tỉa vô ích ngược.

- ◇ **Ý tưởng:** Đánh giá tĩnh (*static eval*) vượt xa biên β một khoảng an toàn lớn $\rightarrow$ Thế trận quá tốt, có thể cắt tỉa nhánh ngay mà không cần tốn công thử nước đi.

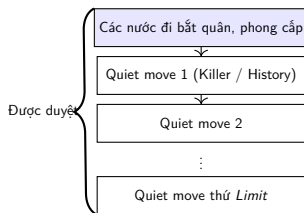


Hình 21: Minh họa Cắt tỉa vô ích ngược.

- ◇ **Ý tưởng:** Ở độ sâu thấp, nếu thử một lượng lớn các nước đi bình thường (*quiet moves*) mà không cải thiện được $\alpha \rightarrow$ Có thể kết luận các nước xếp cuối bảng (chưa được duyệt) đều vô giá trị.

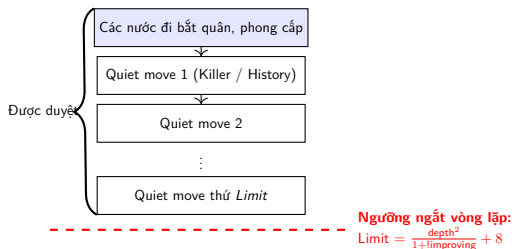
Hình 22: Minh họa cơ chế Cắt tỉa nước đi muộn (LMP).

- ◇ **Ý tưởng:** Ở độ sâu thấp, nếu thử một lượng lớn các nước đi bình thường (*quiet moves*) mà không cải thiện được $\alpha \rightarrow$ Có thể kết luận các nước xếp cuối bảng (chưa được duyệt) đều vô giá trị.



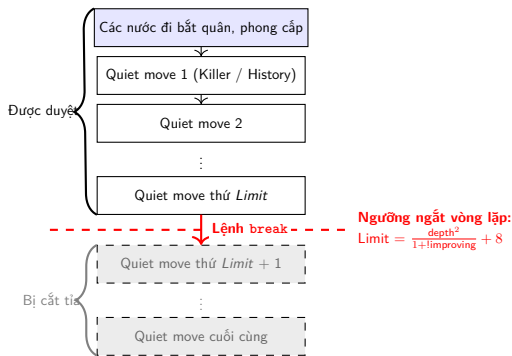
Hình 22: Minh họa cơ chế Cắt tỉa nước đi muộn (LMP).

- ◇ **Ý tưởng:** Ở độ sâu thấp, nếu thử một lượng lớn các nước đi bình thường (*quiet moves*) mà không cải thiện được $\alpha \rightarrow$ Có thể kết luận các nước xếp cuối bảng (chưa được duyệt) đều vô giá trị.



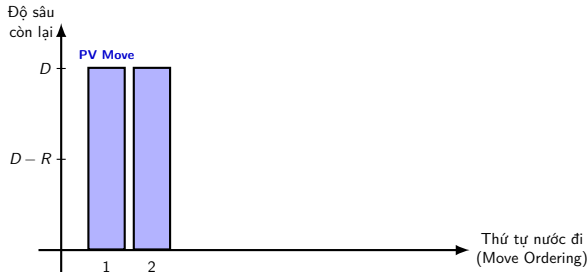
Hình 22: Minh họa cơ chế Cắt tỉa nước đi muộn (LMP).

- ◇ **Ý tưởng:** Ở độ sâu thấp, nếu thử một lượng lớn các nước đi bình thường (*quiet moves*) mà không cải thiện được $\alpha \rightarrow$ Có thể kết luận các nước xếp cuối bảng (chưa được duyệt) đều vô giá trị.



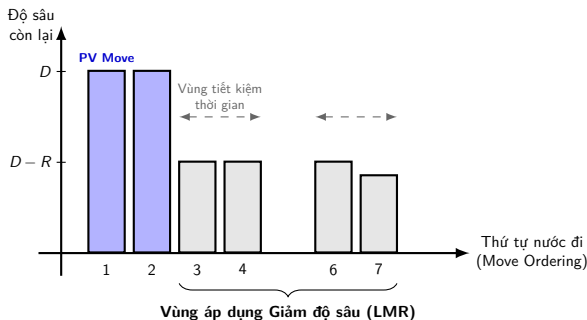
Hình 22: Minh họa cơ chế Cắt tỉa nước đi muộn (LMP).

- ◇ **Ý tưởng:** Các nước đi trễ (kém hấp dẫn) không bị ngắt hoàn toàn bởi LMP sẽ tiếp tục được duyệt nhưng với **độ sâu giảm bớt**.



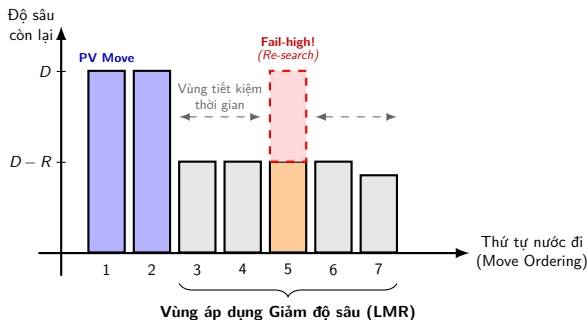
Hình 23: Minh họa tác động của LMR lên cây tìm kiếm

- ◇ **Ý tưởng:** Các nước đi trễ (kém hấp dẫn) không bị ngắt hoàn toàn bởi LMP sẽ tiếp tục được duyệt nhưng với **độ sâu giảm bớt**.



Hình 23: Minh họa tác động của LMR lên cây tìm kiếm

- ◇ **Ý tưởng:** Các nước đi trễ (kém hấp dẫn) không bị ngắt hoàn toàn bởi LMP sẽ tiếp tục được duyệt nhưng với **độ sâu giảm bớt**.
- ◇ **Ngoại lệ:** Nếu nước đi bị giảm độ sâu lại bất ngờ trả về điểm tốt (*fail-high*), ta bắt buộc phải **duyet lại (re-search)** với độ sâu chuẩn.



Hình 23: Minh họa tác động của LMR lên cây tìm kiếm

Bài Toán

Biểu Diễn Trạng Thái

Sinh Nước Đi

Hàm Đánh Giá

Tìm Kiếm

Thực Nghiệm Và Demo

Kết luận



- ◇ **Mục tiêu:** Đảm bảo tính chính xác tuyệt đối của bộ sinh nước đi (*Move Generator*) - nền tảng quyết định sự đúng luật của engine.

- ◇ **Mục tiêu:** Đảm bảo tính chính xác tuyệt đối của bộ sinh nước đi (*Move Generator*) - nền tảng quyết định sự đúng luật của engine.
- ◇ **Phương pháp (PERFT):** Duyệt cây trò chơi ở độ sâu d từ thể trận ban đầu, đếm tổng số nút lá và đối chiếu với kết quả chuẩn mực trên Internet.

- ◇ **Mục tiêu:** Đảm bảo tính chính xác tuyệt đối của bộ sinh nước đi (*Move Generator*) - nền tảng quyết định sự đúng luật của engine.
- ◇ **Phương pháp (PERFT):** Duyệt cây trò chơi ở độ sâu d từ thể trận ban đầu, đếm tổng số nút lá và đối chiếu với kết quả chuẩn mực trên Internet.

Độ sâu	Số nút	Chiều	Ăn quân	Chiều bí
1	44	0	2	0
2	1,920	6	72	0
3	79,666	384	3,159	0
4	3,290,240	19,380	115,365	0
5	133,312,995	953,251	4,917,734	0
6	5,392,831,844	39,288,662	185,194,510	584

Bảng 2: Kết quả PERFT của Lingine tại thể cờ khởi đầu

- ◇ **Kiến trúc tối ưu:** Sử dụng bàn cờ *Bitboard* và kỹ thuật tra cứu bảng tĩnh (*Magic Bitboards*).

- ◇ **Kiến trúc tối ưu:** Sử dụng bàn cờ *Bitboard* và kỹ thuật tra cứu bảng tĩnh (*Magic Bitboards*).
- ◇ **Đo lường thực tế:**
 - **Khối lượng:** Duyệt hơn **5.3 tỷ** trạng thái (độ sâu $d = 6$).
 - **Thời gian:** Hoàn thành trong vôn vện **54.65 giây** (Intel Core i9-13900HX).

- ◇ **Kiến trúc tối ưu:** Sử dụng bàn cờ *Bitboard* và kỹ thuật tra cứu bảng tĩnh (*Magic Bitboards*).
- ◇ **Đo lường thực tế:**
 - **Khối lượng:** Duyệt hơn **5.3 tỷ** trạng thái (độ sâu $d = 6$).
 - **Thời gian:** Hoàn thành trong vòng vọt **54.65 giây** (Intel Core i9-13900HX).
- ◇ **Tốc độ trung bình:**

≈ 98.68 triệu nút / giây

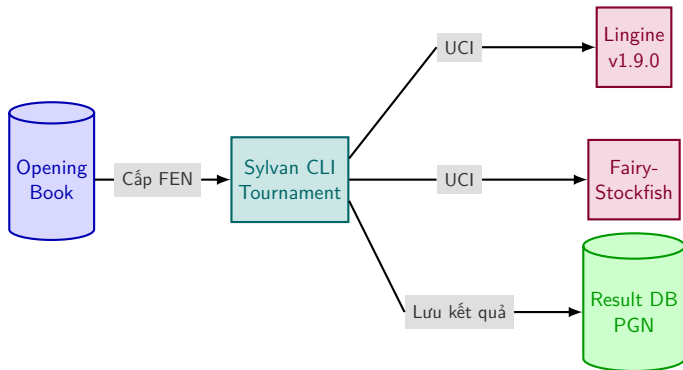
- ◇ **Kiến trúc tối ưu:** Sử dụng bàn cờ *Bitboard* và kỹ thuật tra cứu bảng tĩnh (*Magic Bitboards*).
- ◇ **Đo lường thực tế:**
 - **Khối lượng:** Duyệt hơn **5.3 tỷ** trạng thái (độ sâu $d = 6$).
 - **Thời gian:** Hoàn thành trong vỏn vẹn **54.65 giây** (Intel Core i9-13900HX).
- ◇ **Tốc độ trung bình:**

≈ 98.68 triệu nút / giây

- ◇ **Kết luận:** Loại bỏ hoàn toàn vòng lặp quét tia truyền thống, thay bằng phép toán thao tác bit song song trên thanh ghi 64-bit → Đạt hiệu năng cực hạn.

- ◇ **Hệ thống kiểm thử:** Sử dụng các scripts (`run_match.py`, `run_gauntlet.py`) và trình quản lý giải đấu `sylvan-cli` qua giao thức UCI.

- ◇ **Hệ thống kiểm thử:** Sử dụng các scripts (`run_match.py`, `run_gauntlet.py`) và trình quản lý giải đấu `sylvan-cli` qua giao thức UCI.



Kiểm định nghiêm ngặt qua 3 giai đoạn độc lập:

1. **Gauntlet Performance:** Đấu 3000 ván với 6 cấp độ *Fairy-Stockfish* (ELO từ 1200 đến 2200). Đổi màu quân luân phiên để triệt tiêu lợi thế đi trước.

Kiểm định nghiêm ngặt qua 3 giai đoạn độc lập:

1. **Gauntlet Performance:** Đấu 3000 ván với 6 cấp độ *Fairy-Stockfish* (ELO từ 1200 đến 2200). Đổi màu quân luân phiên để triệt tiêu lợi thế đi trước.
2. **Neighbor Match:** Đấu 1000 ván với phiên bản tiền nhiệm (N vs $N - 1$) đo lường mức cải tiến ELO trực tiếp.

Kiểm định nghiêm ngặt qua 3 giai đoạn độc lập:

1. **Gauntlet Performance:** Đấu 3000 ván với 6 cấp độ *Fairy-Stockfish* (ELO từ 1200 đến 2200). Đổi màu quân luân phiên để triệt tiêu lợi thế đi trước.
2. **Neighbor Match:** Đấu 1000 ván với phiên bản tiền nhiệm (N vs $N - 1$) đo lường mức cải tiến ELO trực tiếp.
3. **Base Match:** Đấu 1000 ván với bản sơ khai (1.0.0) đo tổng tiến bộ lũy kế.

Kiểm định nghiêm ngặt qua 3 giai đoạn độc lập:

1. **Gauntlet Performance:** Đấu 3000 ván với 6 cấp độ *Fairy-Stockfish* (ELO từ 1200 đến 2200). Đổi màu quân luân phiên để triệt tiêu lợi thế đi trước.
2. **Neighbor Match:** Đấu 1000 ván với phiên bản tiền nhiệm (N vs $N - 1$) đo lường mức cải tiến ELO trực tiếp.
3. **Base Match:** Đấu 1000 ván với bản sơ khai (1.0.0) đo tổng tiến bộ lũy kế.

Cấu hình môi trường:

- ◇ *Kiểm soát thời gian:* 3 giây + 0.03 giây/nước.
- ◇ *Đa dạng hóa:* Dùng *Masters Opening Database* (40,711 ván) tạo thế cờ ngẫu nhiên, tránh lặp ván.

Bảng tổng hợp tiến trình 15 phiên bản

Phiên bản	ELO ước lượng	Diff/Trước	Diff/Gốc	Các cải tiến cốt lõi
1.0.0	1465 ± 8.9	N/A	N/A	Sinh nước đi Bitboard, Negamax Fail-soft, Quiescence Search.

Bảng 3: Tiến trình phát triển và tăng trưởng sức mạnh ELO



Bảng tổng hợp tiến trình 15 phiên bản

Phiên bản	ELO ước lượng	Diff/Trước	Diff/Gốc	Các cải tiến cốt lõi
1.0.0	1465 ± 8.9	N/A	N/A	Sinh nước đi Bitboard, Negamax Fail-soft, Quiescence Search.
1.1.0	1460 ± 8.8	-9.0	-2.1	Hoàn thiện luật cờ tướng (Cản Mã, Tượng lộ mặt).

Bảng 3: Tiến trình phát triển và tăng trưởng sức mạnh ELO



Bảng tổng hợp tiến trình 15 phiên bản

Phiên bản	ELO ước lượng	Diff/Trước	Diff/Gốc	Các cải tiến cốt lõi
1.0.0	1465 ± 8.9	N/A	N/A	Sinh nước đi Bitboard, Negamax Fail-soft, Quiescence Search.
1.1.0	1460 ± 8.8	-9.0	-2.1	Hoàn thiện luật cờ tướng (Cản Mã, Tượng lộ mặt).
1.2.0	1567 ± 8.7	+96.2	+85.0	Transposition Table (TT).

Bảng 3: Tiến trình phát triển và tăng trưởng sức mạnh ELO



Bảng tổng hợp tiến trình 15 phiên bản

Phiên bản	ELO ước lượng	Diff/Trước	Diff/Gốc	Các cải tiến cốt lõi
1.0.0	1465 ± 8.9	N/A	N/A	Sinh nước đi Bitboard, Negamax Fail-soft, Quiescence Search.
1.1.0	1460 ± 8.8	-9.0	-2.1	Hoàn thiện luật cờ tướng (Cản Mã, Tượng lộ mặt).
1.2.0	1567 ± 8.7	+96.2	+85.0	Transposition Table (TT).
1.3.0	1572 ± 8.7	+20.2	+100.3	Aspiration Window.

Bảng 3: Tiến trình phát triển và tăng trưởng sức mạnh ELO



Bảng tổng hợp tiến trình 15 phiên bản

Phiên bản	ELO ước lượng	Diff/Trước	Diff/Gốc	Các cải tiến cốt lõi
1.0.0	1465 ± 8.9	N/A	N/A	Sinh nước đi Bitboard, Negamax Fail-soft, Quiescence Search.
1.1.0	1460 ± 8.8	-9.0	-2.1	Hoàn thiện luật cờ tướng (Cản Mã, Tượng lộ mặt).
1.2.0	1567 ± 8.7	+96.2	+85.0	Transposition Table (TT).
1.3.0	1572 ± 8.7	+20.2	+100.3	Aspiration Window.
1.4.0	1612 ± 8.6	+69.4	+145.5	Killer Moves, History Heuristics.

Bảng 3: Tiến trình phát triển và tăng trưởng sức mạnh ELO



Phiên bản	ELO ước lượng	Diff/Trước	Diff/Gốc	Các cải tiến cốt lõi
1.0.0	1465 ± 8.9	N/A	N/A	Sinh nước đi Bitboard, Negamax Fail-soft, Quiescence Search.
1.1.0	1460 ± 8.8	-9.0	-2.1	Hoàn thiện luật cờ tướng (Cản Mã, Tượng lộ mặt).
1.2.0	1567 ± 8.7	+96.2	+85.0	Transposition Table (TT).
1.3.0	1572 ± 8.7	+20.2	+100.3	Aspiration Window.
1.4.0	1612 ± 8.6	+69.4	+145.5	Killer Moves, History Heuristics.
1.5.0	1854 ± 8.7	+358.8	+556.7	Piece-Square Tables (PST).

Bảng 3: Tiến trình phát triển và tăng trưởng sức mạnh ELO

Bảng tổng hợp tiến trình 15 phiên bản

Phiên bản	ELO ước lượng	Diff/Trước	Diff/Gốc	Các cải tiến cốt lõi
1.0.0	1465 ± 8.9	N/A	N/A	Sinh nước đi Bitboard, Negamax Fail-soft, Quiescence Search.
1.1.0	1460 ± 8.8	-9.0	-2.1	Hoàn thiện luật cờ tướng (Cản Mã, Tượng lộ mặt).
1.2.0	1567 ± 8.7	+96.2	+85.0	Transposition Table (TT).
1.3.0	1572 ± 8.7	+20.2	+100.3	Aspiration Window.
1.4.0	1612 ± 8.6	+69.4	+145.5	Killer Moves, History Heuristics.
1.5.0	1854 ± 8.7	+358.8	+556.7	Piece-Square Tables (PST).
1.6.0	1916 ± 8.8	+80.6	+640.0	Search Extensions (Check, Singular, One-Reply).
1.6.1 - 1.6.5	1920	Biến động	600	Tái cấu trúc, tối ưu hóa TT trong Q-Search.

Bảng 3: Tiến trình phát triển và tăng trưởng sức mạnh ELO



Bảng tổng hợp tiến trình 15 phiên bản

Phiên bản	ELO ước lượng	Diff/Trước	Diff/Gốc	Các cải tiến cốt lõi
1.0.0	1465 $\pm$ 8.9	N/A	N/A	Sinh nước đi Bitboard, Negamax Fail-soft, Quiescence Search.
1.1.0	1460 $\pm$ 8.8	-9.0	-2.1	Hoàn thiện luật cờ tướng (Cản Mã, Tượng lộ mặt).
1.2.0	1567 $\pm$ 8.7	+96.2	+85.0	Transposition Table (TT).
1.3.0	1572 $\pm$ 8.7	+20.2	+100.3	Aspiration Window.
1.4.0	1612 $\pm$ 8.6	+69.4	+145.5	Killer Moves, History Heuristics.
1.5.0	1854 $\pm$ 8.7	+358.8	+556.7	Piece-Square Tables (PST).
1.6.0	1916 $\pm$ 8.8	+80.6	+640.0	Search Extensions (Check, Singular, One-Reply).
1.6.1 - 1.6.5	1920	Biến động	600	Tái cấu trúc, tối ưu hóa TT trong Q-Search.
1.7.0	2027 $\pm$ 9.2	+152.2	+536.9	Tapered Eval, PVS, LMR, Null Move Pruning.

Bảng 3: Tiến trình phát triển và tăng trưởng sức mạnh ELO



Bảng tổng hợp tiến trình 15 phiên bản

Phiên bản	ELO ước lượng	Diff/Trước	Diff/Gốc	Các cải tiến cốt lõi
1.0.0	1465 $\pm$ 8.9	N/A	N/A	Sinh nước đi Bitboard, Negamax Fail-soft, Quiescence Search.
1.1.0	1460 $\pm$ 8.8	-9.0	-2.1	Hoàn thiện luật cờ tướng (Cản Mã, Tượng lộ mặt).
1.2.0	1567 $\pm$ 8.7	+96.2	+85.0	Transposition Table (TT).
1.3.0	1572 $\pm$ 8.7	+20.2	+100.3	Aspiration Window.
1.4.0	1612 $\pm$ 8.6	+69.4	+145.5	Killer Moves, History Heuristics.
1.5.0	1854 $\pm$ 8.7	+358.8	+556.7	Piece-Square Tables (PST).
1.6.0	1916 $\pm$ 8.8	+80.6	+640.0	Search Extensions (Check, Singular, One-Reply).
1.6.1 - 1.6.5	1920	Biến động	600	Tái cấu trúc, tối ưu hóa TT trong Q-Search.
1.7.0	2027 $\pm$ 9.2	+152.2	+536.9	Tapered Eval, PVS, LMR, Null Move Pruning.
1.8.0	2189 $\pm$ 10.3	+237.6	+619.4	Mobility/Defender Bonus, Texel Tuning.

Bảng 3: Tiến trình phát triển và tăng trưởng sức mạnh ELO



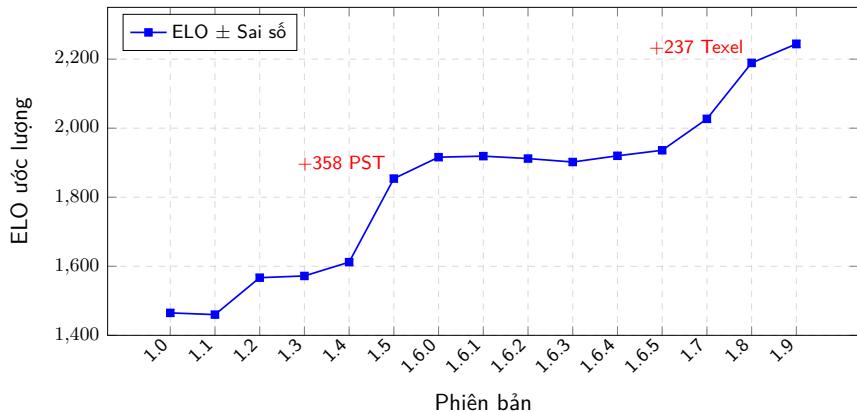
Bảng tổng hợp tiến trình 15 phiên bản

Phiên bản	ELO ước lượng	Diff/Trước	Diff/Gốc	Các cải tiến cốt lõi
1.0.0	1465 $\pm$ 8.9	N/A	N/A	Sinh nước đi Bitboard, Negamax Fail-soft, Quiescence Search.
1.1.0	1460 $\pm$ 8.8	-9.0	-2.1	Hoàn thiện luật cờ tướng (Cản Mã, Tướng lộ mặt).
1.2.0	1567 $\pm$ 8.7	+96.2	+85.0	Transposition Table (TT).
1.3.0	1572 $\pm$ 8.7	+20.2	+100.3	Aspiration Window.
1.4.0	1612 $\pm$ 8.6	+69.4	+145.5	Killer Moves, History Heuristics.
1.5.0	1854 $\pm$ 8.7	+358.8	+556.7	Piece-Square Tables (PST).
1.6.0	1916 $\pm$ 8.8	+80.6	+640.0	Search Extensions (Check, Singular, One-Reply).
1.6.1 - 1.6.5	1920	Biến động	600	Tái cấu trúc, tối ưu hóa TT trong Q-Search.
1.7.0	2027 $\pm$ 9.2	+152.2	+536.9	Tapered Eval, PVS, LMR, Null Move Pruning.
1.8.0	2189 $\pm$ 10.3	+237.6	+619.4	Mobility/Defender Bonus, Texel Tuning.
1.9.0	2244 $\pm$ 10.9	+127.4	+651.3	RFP, Futility Pruning, LMP, IIR.

Bảng 3: Tiến trình phát triển và tăng trưởng sức mạnh ELO



Biểu đồ tăng trưởng ELO



Hình 24: Biểu đồ tăng trưởng ELO của Lingine qua 15 phiên bản

Bài Toán

Biểu Diễn Trạng Thái

Sinh Nước Đi

Hàm Đánh Giá

Tìm Kiếm

Thực Nghiệm Và Demo

Kết luận



◇ 1. Hiệu ứng chân trời (Horizon Effect):

- *Vấn đề*: Engine thí quân vô nghĩa để đẩy đòn bắt lợi ra khỏi "chân trời" tìm kiếm, dẫn đến đánh giá sai lầm.
- *Giải pháp*: Triển khai **Quiescence Search** kết hợp Check Extensions. Tiếp tục duyệt các nước ăn quân đến khi thể trận tĩnh hoàn toàn.

◇ 1. Hiệu ứng chân trời (Horizon Effect):

- *Vấn đề*: Engine thí quân vô nghĩa để đẩy đòn bắt lợi ra khỏi "chân trời" tìm kiếm, dẫn đến đánh giá sai lầm.
- *Giải pháp*: Triển khai **Quiescence Search** kết hợp Check Extensions. Tiếp tục duyệt các nước ăn quân đến khi thế trận tĩnh hoàn toàn.

◇ 2. Tranh chấp đồng bộ hóa (UCI):

- *Vấn đề*: Lệnh ngắt stop từ GUI gửi đến bất kỳ lúc nào. Khóa Mutex truyền thống gây nghẽn hoặc treo luồng.
- *Giải pháp*: Dùng biến nguyên tử **AtomicBool** chia sẻ giữa luồng đọc và luồng tìm kiếm → Dừng tức thời (< 1 ms) không chặn luồng.

Dự án được xây dựng trên nền tảng công nghệ mã nguồn mở:

- ◇ **Ngôn ngữ & Công cụ: Rust** (rustc, cargo). Biên dịch tối ưu với `opt-level=3` và `target-cpu=native`.

Dự án được xây dựng trên nền tảng công nghệ mã nguồn mở:

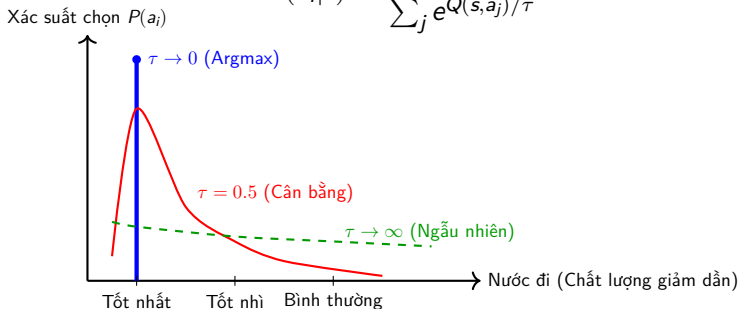
- ◇ **Ngôn ngữ & Công cụ: Rust** (`rustc`, `cargo`). Biên dịch tối ưu với `opt-level=3` và `target-cpu=native`.
- ◇ **Quản lý bộ nhớ:** Thư viện `arrayvec` lưu danh sách nước đi trực tiếp trên bộ nhớ Stack, loại bỏ overhead cấp phát Heap động.

Dự án được xây dựng trên nền tảng công nghệ mã nguồn mở:

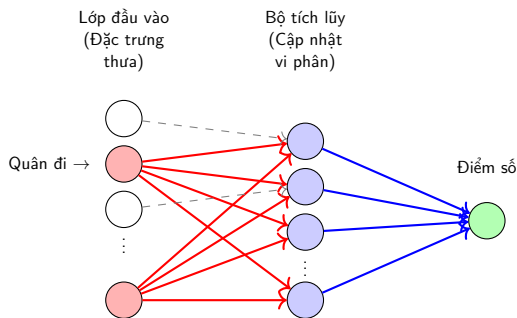
- ◇ **Ngôn ngữ & Công cụ: Rust** (rustc, cargo). Biên dịch tối ưu với `opt-level=3` và `target-cpu=native`.
- ◇ **Quản lý bộ nhớ:** Thư viện `arrayvec` lưu danh sách nước đi trực tiếp trên bộ nhớ Stack, loại bỏ overhead cấp phát Heap động.
- ◇ **Hệ sinh thái Kiểm thử:**
 - **Python Suite:** Viết kịch bản tự động hóa giải đấu.
 - **Sylvan CLI & GUI:** Giao diện điều phối qua giao thức UCI.
 - **Đổi chiều & Benchmark:** *Fairy-Stockfish*, *Pikafish* (đo Elo) và kiến trúc *Reckless*, *Viridithas* (tham khảo cấu trúc).

- ◇ **Vấn đề:** Việc luôn chọn nước đi tốt nhất (bestmove) khiến lối chơi rập khuôn, dễ bị đối thủ "bắt bài".
- ◇ **Giải pháp:** Chuyển điểm số thành phân phối xác suất dựa trên nhiệt độ τ để đa dạng hóa nước đi:

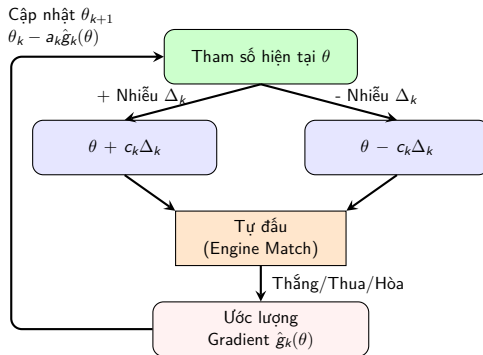
$$P(a_i|s) = \frac{e^{Q(s,a_i)/\tau}}{\sum_j e^{Q(s,a_j)/\tau}}$$



- ◇ Mạng NNUE (Efficiently Updatable Neural Networks) giúp nắm bắt tương tác phi tuyến (ví dụ: Xe-Pháo-Mã).
- ◇ **Điểm mạnh:** Cập nhật vi phân. Chỉ tính lại trọng số cho quân cờ vừa di chuyển, giữ vững tốc độ duyệt NPS.



- ◇ **Vấn đề:** Hàng trăm tham số cần tìm kiếm (LMR, NMP) đang được chỉnh tay, không thể dùng Gradient Descent vì đây là hộp đen.
- ◇ **Giải pháp:** Dùng thuật toán **SPSA**. Tạo nhiễu ngẫu nhiên lên toàn bộ tham số, cho engine tự đấu (Self-play) để ước lượng Gradient.



Cảm ơn mọi người đã lắng nghe!

Q & A

